OpenAI Codex

Codex CLI 使用教程

从第一次上手，到形成稳定工作流

这是一份面向初学者的中文教程。它不追求“参数大全”，而是把安装、提问、权限、审改动、自动化和排错串成一条真正能落地的使用路径。

适合读者

会一点命令行，但第一次系统使用 Codex CLI 的开发者。

阅读重点

真实工作流、提示词写法、权限边界、Git 与 CI（持续集成）配合。

交付目标

看完就能独立完成一轮“理解项目、定位问题、做小改动、复核结果”。

本文根据本地 `codex --help` 输出与 OpenAI 官方 Codex 文档整理；请以你当前安装版本为准。

目录

教程大纲 8

1 Codex CLI 是什么 9

1.1 先看一张工作流程图 9

1.2 一个很实际的场景 10

2 安装前准备 11

2.1 你至少要准备什么 11

2.2 安装前建议先确认目录 11

2.3 再确认一下你的使用目标 12

3 安装、登录与首次验证 13

3.1 先装上 13

3.2 登录有两条主路 13

3.3 方式一：用 ChatGPT 登录 13

3.4 方式二：用 API 密钥登录 14

3.5 无浏览器环境怎么办 14

3.6 Linux 环境下最常见的三种起步方式 14

3.7 macOS、Windows 和 WSL 用户额外看这几眼 15

3.8 第一次验证，不要跳过 16

3.9 升级也很简单 16

4 基本使用流程 17

4.1 典型使用过程步骤图 17

4.2 第一次提问，建议这样说 17

4.3 追问时，尽量把范围收紧 18

5 常见命令与操作方式 19

5.1 先看一张分类图 19

5.2 交互式入口：`codex` 19

5.3 最基础的帮助与状态 19

5.4 非交互执行：codex exec 20
5.5 代码评审：codex review 20
5.6 恢复、分叉、应用结果 20
5.7 两个高频参数 20
5.8 什么时候该开 --search，什么时候只看本地代码 21

6 权限、沙箱与审批机制 22

6.1 先记住三个沙箱级别 22
6.2 再记住三个审批策略 22
6.3 最常见的组合 23
6.4 为什么很多人推荐从 workspace-write 开始 23
6.5 什么时候会要求你批准 23
6.6 一个判断思路图 24

7 实用斜杠命令与会话控制 25

7.1 先看这张入口选择图 25
7.2 最值得先记的几条 25
7.3 /status：先确认它到底在什么状态 26
7.4 /permissions：会话里随时改权限 26
7.5 /review 和 /diff：改完一定要连着看 26
7.6 /plan 和 /init：一个管当前任务，一个管长期规则 27
7.7 /compact 和 /mention：上下文太长时特别有用 27
7.8 /model：什么时候值得切模型 27
7.9 /new 和 /clear：在当前终端里重开一轮 27
7.10 /resume 和 /fork：恢复旧会话，或者从旧会话分叉 28
7.11 什么时候退出后重开 codex? 28
7.12 会话命令对照图 28

8 最小可运行示例 30

8.1 示例目标 30
8.2 一轮最小示例 30
8.3 最后别忘了做一次会话内检查 31

9 三轮对话示例：怎么把问题越问越准 **32**
9.1 第一轮：先把地图拿到手 32
9.2 第二轮：把范围压到功能级 32
9.3 第三轮：要求它给可交付结果 33

10 高质量提示词模板库 **34**
10.1 模板一：看懂陌生项目 34
10.2 模板二：定位某个功能 34
10.3 模板三：修一个明确的问题 34
10.4 模板四：做一个小改动，但限制范围 35
10.5 模板五：做代码审查 35
10.6 模板六：让它先采访你 35

11 几个典型使用场景 **36**
11.1 场景一：接手陌生项目 36
11.2 场景二：定位某个功能 36
11.3 场景三：解释报错 36
11.4 场景四：改一个小功能 36
11.5 场景五：提交前做一次代码审查 37

12 和 Git、测试、非交互模式的配合 **38**
12.1 和 Git 的最小配合方式 38
12.2 改坏了怎么办：先保留现场，再决定怎么撤 38
12.3 改完以后，别只看它说了什么 39
12.4 如果你知道测试命令，最好明确告诉它 39
12.5 把长期规则写进 `AGENTS.md` 40
12.6 除了 `AGENTS.md`，也值得把常用设置固化进 `config.toml` 40
12.7 非交互执行：什么时候该用 `codex exec` 41
12.8 CI 里怎么认证 41
12.9 把 `exec` 真正接进脚本：人看摘要，机器看结构 41
12.10 为什么 `exec` 默认要求 Git 仓库 42

13 任务完成验收卡 **44**

14 常见错误与排查思路 **45**
14.1 排查思路图 45
14.2 问题一：命令找不到 45
14.3 问题二：明明在项目里，却读不到对的内容 46
14.4 问题三：回答太空泛 46
14.5 问题四：它总在问你要不要批准 46
14.6 问题五：脚本里跑不通 46

15 使用建议与最佳实践 **48**
15.1 建议一：任务要具体 48
15.2 建议二：先小后大 48
15.3 建议三：让它输出结构化结果 48
15.4 建议四：让它知道怎么验证 49
15.5 建议五：复杂任务先计划 49
15.6 建议六：把重复规则沉淀下来 49

16 新手最容易踩的坑 **50**

17 高风险任务边界清单 **51**

18 进阶使用技巧：从会用到会控场 **53**
18.1 技巧一：先做方案评审，不急着改代码 53
18.2 技巧二：大仓库先控上下文，不要一口气全拉进来 53
18.3 技巧三：用 `/fork` 保留旧思路，同时试另一条线 54
18.4 技巧四：用 `/model` 做快慢分工 54
18.5 技巧五：先定义成功标准，再开始动手 55
18.6 技巧六：把代码审查做成闭环，不是只跑一次 55
18.7 技巧七：固定结构化交付格式 56
18.8 技巧八：把 `codex exec` 用在真正适合自动化的地方 56
18.9 技巧九：先用一个小例子理解 MCP 到底值在哪 57

19 正文收束：3 分钟速读总结 **59**

20 常用命令速查表 **60**

21 新手避坑清单 **61**

22 附录：从零到一演练页 **62**

22.1 目标 62

22.2 第 1 步：把公开仓库拉到本地 62

22.3 第 2 步：第一次进入会话 62

22.4 第 3 步：定位一个具体点 63

22.5 第 4 步：做一个低风险改动 63

22.6 第 5 步：复核改动 63

22.7 第 6 步：完成一轮最小闭环 64

23 附录：练习题怎么看算合格 **65**

23.1 练习一：合格结果至少要有路径和顺序 65

23.2 练习二：合格结果要讲清调用链，不只是列文件 65

23.3 练习三：合格结果要控住范围，还要说清怎么验 66

23.4 练习四和练习五：重点看它有没有真的“落地” 66

24 附录：好提示词 vs 坏提示词 **68**

24.1 一个简单判断法 68

24.2 最常见的坏味道 69

25 附录：常见场景小抄 **70**

25.1 场景一：接手项目 70

25.2 场景二：处理报错 70

25.3 场景三：补测试 70

25.4 场景四：做代码审查 71

26 附录：CI 集成示例 **72**

26.1 最小命令行示例 72

26.2 一个适合 CI 的思路 72

26.3 GitHub Actions 风格示例 73

26.4 CI 里最容易踩的坑 73

27 附录：真实排错案例 **74**

27.1 案例背景 74
27.2 第一轮，不要急着让它直接修 74
27.3 第二轮，把范围压到文件级别 75
27.4 第三轮，让它给最小修复方案 75
27.5 这类案例里，Codex 最值得帮你的地方 75
27.6 如果它一上来就想改很多文件怎么办 76
27.7 这页案例真正想教你的东西 76

28 附录：练习题页 77

28.1 练习一：找到阅读入口 77
28.2 练习二：定位一个功能 77
28.3 练习三：做一个低风险小改动 78
28.4 练习四：做一次代码审查 78
28.5 练习五：写一条更好的提示词 79

29 附录：AGENTS.md 示例 80

29.1 什么样的内容最值得写进去 80
29.2 一份适合中小型项目起步的样板 80
29.3 如果你的仓库更复杂，可以再加什么 82
29.4 写 AGENTS.md 的一个实用原则 82

30 附录：版本变更提示框 83

30.1 最推荐的检查顺序 83
30.2 哪些地方最容易发生版本差异 83

31 FAQ 85

31.1 Q1：如果上下文窗口超了，应该怎么办？ 85
31.2 Q2：它回答得很空泛，是模型不行吗？ 85
31.3 Q3：什么时候该开新会话，而不是一直接着问？ 85
31.4 Q4：会话到底怎么切？能像别的工具那样在多个会话间切来切去吗？ . . . 86
31.5 Q5：文档里是不是还有命令没讲？ 86
31.6 Q6：它改完代码后，我最少要做什么检查？ 86

32 附录：完整命令参考 87

32.1 先看顶层命令总览 87
32.2 第一组：主线工作流命令 88
32.3 第二组：扩展、集成和云端命令 91
32.4 第三组：环境、沙箱和调试命令 93
32.5 高频全局参数怎么读 93
32.6 最后给一个查命令的顺序 95

33 官方资源与版本说明 **96**
33.1 当教程和你本地行为不一致时怎么办 96

34 一页式总结页 **98**

教程大纲

- **第一部分：先把上手主线跑通**
  从“Codex CLI 是什么”一路到“最小可运行示例”，先解决安装、登录、进目录、提任务、看权限、看结果这条最短路径。
- **第二部分：把一轮任务做完整**
  用三轮对话、提示词模板、典型场景、Git / 测试 / `exec` 配合、验收卡和排错思路，把“会用”变成“能落地”。
- **第三部分：长期稳定使用**
  这里会讲最佳实践、新手易坑、高风险任务边界，以及“从会用到会控场”的进阶技巧。
- **第四部分：速查与附录**
  正文后面先给速读总结、命令速查表和避坑清单；再按需查练习、演练页、`AGENTS.md`、CI、完整命令参考、FAQ 和官方资源。

提示

第一次通读，建议先把前三部分顺着看完；第四部分更像工具箱，遇到对应场景再翻会更高效。

1

Codex CLI 是什么

这一节学完你能做什么

学完这一节，你会知道 Codex CLI 适合干什么、不适合干什么，以及为什么很多人把它当成“终端里的代码搭档”。

先用一句最朴素的话解释：**Codex CLI 是一个跑在终端里的 AI 编码代理**。你在项目目录里和它对话，它会结合当前代码、目录、命令行环境，帮你读代码、改代码、跑命令、解释报错、做代码审查。

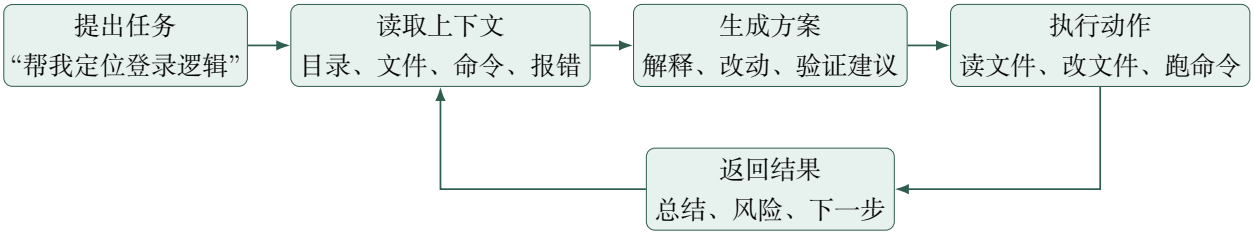
它最适合的事，通常是下面四类：

- 接手陌生项目时，先帮你找入口、找主流程、给阅读顺序。
- 你知道要改哪里，但不想自己先翻几十个文件。
- 你遇到构建失败、测试失败、脚本报错，想先得到一个靠谱的排查顺序。
- 你要补文档、补测试、改小功能，希望先出一个可用初稿。

它不太适合的事，也要提前说清楚：

- 你完全没有目标，只想“让它自动把整个项目做完”。
- 你准备把高风险命令、删除操作、线上变更全交给它，不做复核。
- 你把需求判断、架构取舍、业务责任也一起外包给它。

1.1 先看一张工作流程图



1.2 一个很实际的场景

比如你打开一个陌生仓库，里面有十几个目录。你现在既不知道入口文件在哪，也不知道应该从前端、后端还是脚本开始看。你可以让 Codex CLI：

- 先扫目录结构；
- 再指出入口文件和关键目录；
- 然后按新手视角给阅读顺序；
- 最后在你确认范围后做小改动。

提示

把 Codex CLI 理解成“既能看代码也能动手，但需要你给目标和边界的工程搭档”，这个心智模型最稳。

小结

Codex CLI 的价值，不是神奇自动化，而是把“理解项目、修改项目、解释结果、验证改动”放到一个终端工作流里。

2

安装前准备

这一节学完你能做什么

学完这一节，你会知道在安装前至少要确认哪些东西，避免工具装上了，却因为目录、账号或系统环境卡住。

很多新手的问题不是“不会装”，而是“装完以后不知道为什么不能用”。所以安装前先确认下面几件事。

2.1 你至少要准备什么

- 一台能正常使用终端的电脑。
- 一个你愿意拿来练手的项目目录，最好不是最核心的正式仓库。
- 可用的认证方式：ChatGPT 账号登录，或者 OpenAI API 密钥。
- 基本的命令行习惯，比如会用 `cd`、`ls`、`pwd`。

2.2 安装前建议先确认目录

```
pwd
ls
```

这两条命令看起来很基础，但足够帮你避免很多低级错误：

- 先确认你现在是不是站在目标项目里；
- 再确认这个目录到底有哪些文件和子目录。

注意

如果你还没搞清楚“当前目录是不是目标项目”，先别急着让 Codex 改东西。这是新手最常见的坑之一。

2.3 再确认一下你的使用目标

安装前最好先回答自己一个问题：你现在主要是想用它做什么？

- 只是想交互式地看项目、改小功能。
- 想把某些固定任务做成脚本或 CI（Continuous Integration，持续集成）。
- 想在本地终端里配合 Git 做评审和修复。

不同目标会影响你后面更常用的功能。比如日常开发更常用 `codex`、`/diff`、`/review`；脚本化更常用 `codex exec`。

小结

安装前先确认三件事：目录、认证方式、使用目标。先把“在哪里用、怎么登、打算拿它做什么”说清楚，后面会顺很多。

3

安装、登录与首次验证

这一节学完你能做什么

学完这一节，你会知道怎么把 Codex CLI 真的装起来、登录进去，并完成第一次最小验证。

3.1 先装上

OpenAI 官方 Codex CLI 页面当前给出了 `npm` 安装入口。下面用最常见的方式举例：

```
npm i -g @openai/codex
```

装完后可以先确认命令是否存在：

```
codex --version
codex --help
```

如果 `--help` 能正常输出，说明至少两件事成立了：

- CLI 已经安装成功；
- 当前终端能找到 `codex` 这个命令。

3.2 登录有两条主路

官方文档当前写得很明确：Codex 支持两种 OpenAI 登录方式。

- 用 ChatGPT 登录：适合日常交互式使用，是 CLI 默认的登录路径。
- 用 API 密钥登录：适合按 API 计费的使用方式，也常用于自动化和 CI。

3.3 方式一：用 ChatGPT 登录

最直接的方式通常就是先跑：

```
codex
```

第一次运行时，CLI 会提示你登录。按官方文档，默认会走 ChatGPT 登录流程，浏览器会打开登录页，完成后再回到终端。

如果你想显式发起登录，也可以：

```
codex login
codex login status
```

3.4 方式二：用 API 密钥登录

如果你更希望按 API 使用方式来计费，或者你就是打算后面接脚本，那 API 密钥登录很实用。本地 CLI 帮助里给出了这类写法：

```
printenv OPENAI_API_KEY | codex login --with-api-key
```

它的意思很简单：把环境变量里的密钥通过标准输入传给登录命令。

这里顺手区分一下两个常见变量名：`OPENAI_API_KEY` 常出现在“把密钥传给 `codex login --with-api-key`”这类登录示例里；到了 `codex exec` 和 CI 场景，官方更常用的是 `CODEX_API_KEY`。

注意

别把 API 密钥直接写进教程截图、聊天记录或仓库文件里。环境变量是最基本也最稳妥的做法。

3.5 无浏览器环境怎么办

如果你在远端机器、容器或者没有图形界面的环境里工作，官方文档给了两条思路：

- 设备码登录：运行 `codex login --device-auth`，按提示在浏览器里完成一次性验证。
- 在本地登录后，把认证缓存复制过去。

官方文档还特别提醒：如果你使用文件形式缓存认证信息，`~/.codex/auth.json` 需要当成密码对待，绝对不要提交到仓库。

3.6 Linux 环境下最常见的三种起步方式

下面默认你用的是 Linux 终端。对新手来说，最常见的其实就是三种环境：

- **本地图形界面 Linux**：最省心，直接在项目目录里运行 `codex`，按提示登录即可。
- **远端 Linux / SSH**：通常没有浏览器，优先用 `codex login --device-auth`，或者先在本地登录，再把认证缓存安全地带过去。

- **容器或云主机里的 Linux**：除了登录，还要确认当前挂载目录是不是你真想操作的工作区，以及容器销毁后认证信息会不会丢。

不管是哪一种，开始前都建议先跑一组最小检查：

```
pwd
ls
codex --help
codex login status
```

这四步的作用分别很实际：

- `pwd`：确认你现在到底站在哪个目录；
- `ls`：确认眼前是不是你预期的项目；
- `codex --help`：确认 CLI 本身可用；
- `codex login status`：确认认证状态是不是已经就绪。

提示

如果你在远端 Linux 上工作，最容易忽略的不是命令本身，而是“我现在到底在哪个目录、这个目录是不是可写、认证是不是还在”。先把这三件事确认掉，后面会顺很多。

3.7 macOS、Windows 和 WSL 用户额外看这儿眼

如果你不是在纯 Linux 环境里工作，也完全没问题。真正容易卡住的，不是 Codex 本身，而是终端、路径、认证和项目目录混在一起了。

- **macOS**：整体体验和 Linux 很接近。重点确认当前终端是不是你常用的 shell，`codex` 能不能被 PATH 找到，以及你打开的是不是正确的项目目录。
- **Windows PowerShell**：命令能跑，但环境变量写法和 Bash 不一样。临时设置 API 密钥时，更像下面这样：

```
$env:OPENAI_API_KEY="your_api_key"
$env:CODEX_API_KEY="your_api_key"
```

- **WSL**：最好把它当成“一台单独的 Linux 环境”来理解。认证、Git、Node、测试命令，尽量都在同一个 WSL 终端里完成。

如果你主要在 WSL 里开发，一个很稳的顺序通常是：

1. 把项目放在你真正准备开发的那一侧；
2. 在那一侧完成登录、运行 `codex`、跑 Git 和测试；
3. 不要一会儿在 Windows 终端里改，一会儿又切去 WSL 里验证。

注意

最容易出问题的场景，是“仓库在 WSL 里，终端却在 Windows 那边；或者认证在一边，Git 和依赖在另一边”。如果项目主要跑在 WSL，就尽量把 Codex、Git、测试也都放在 WSL 里统一完成。

3.8 第一次验证，不要跳过

建议按下面顺序做一次最小验证：

1. `codex --help` 能输出；
2. `codex login status` 能确认已登录；
3. 进入一个项目目录后，运行 `codex`；
4. 提一个最小任务，看看它能不能正确读取项目结构。

3.9 升级也很简单

官方 CLI 页面当前给出的 npm 升级方式是：

```
npm i -g @openai/codex@latest
```

提示

新版本发布会比较频繁。你如果发现教程里的某个选项和本地帮助不一致，优先相信你机器上的 `codex --help`。

小结

这一章最重要的不是“背命令”，而是形成一个稳定顺序：安装、登录、验证。只要这三步走通，后面的使用成本会大幅下降。

4

基本使用流程

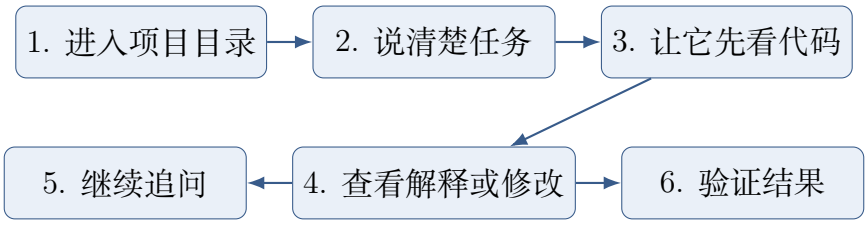
这一节学完你能做什么

学完这一节，你会知道第一次真正使用 Codex CLI 的标准节奏，至少能独立完成一轮“提任务 -> 看结果 -> 继续追问 -> 做验证”。

对新手来说，最容易上手的不是复杂参数，而是一个简单的工作流：

1. 进入项目目录；
2. 用一句清楚的话描述任务；
3. 先让它读上下文；
4. 看它的解释、计划或改动建议；
5. 不满意就继续追问；
6. 改完以后自己做验证。

4.1 典型使用过程步骤图



4.2 第一次提问，建议这样说

比起一句“帮我看看这个项目”，下面这种说法通常更有效：

```
请先看一下当前项目的目录结构，告诉我入口文件、主要模块，以及新手应该先看哪几个文件。
```

4.3 追问时，尽量把范围收紧

如果第一轮结果太泛，你可以继续这样问：

```
请不要泛泛介绍，重点解释登录相关代码在哪、数据流怎么走，以及我应该先从哪个文件开始读。
```

注意

“帮我优化一下”“帮我改好”这种说法太宽，容易得到看起来很忙、但不够落地的结果。新手阶段尽量把目标说具体。

小结

第一次用 Codex CLI，不求一步到位。先学会把任务说清楚，再学会继续追问，这比背一堆参数更重要。

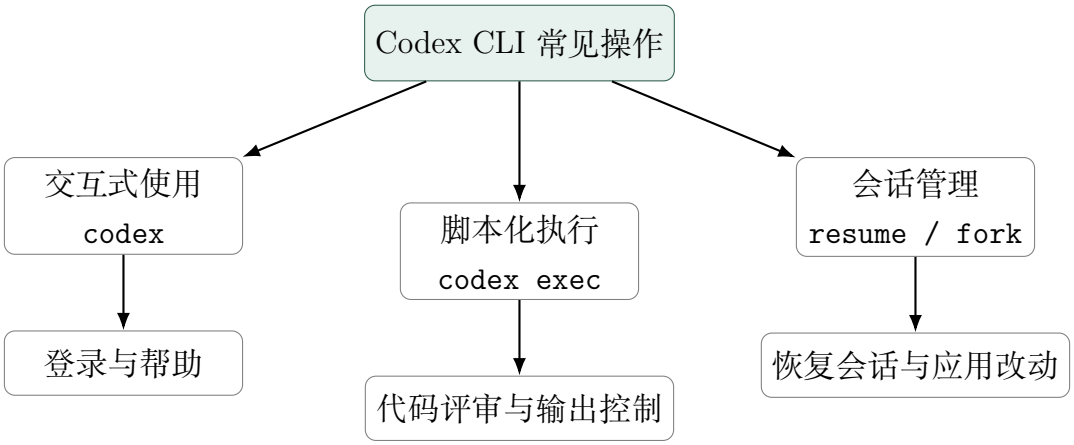
5

常见命令与操作方式

这一节学完你能做什么

学完这一节，你会知道日常最常用的命令有哪些，各自适合什么场景。

5.1 先看一张分类图



5.2 交互式入口：codex

```
codex
```

这是最常见的入口。适合下面这些场景：

- 你要围绕当前项目连续提问；
- 你想边看解释边调整方向；
- 你希望它先分析，再改动，再复盘。

5.3 最基础的帮助与状态

```
codex --help
codex --version
codex login status
```

忘了支持什么功能、当前版本是什么、是否已经登录，先看这些，比乱猜靠谱。

5.4 非交互执行：codex exec

```
codex exec "检查当前仓库里最可能导致测试失败的改动"
```

这类命令更适合脚本、流水线、一次性任务。官方文档也明确把 codex exec 定位为“脚本化或 CI 风格的非交互执行模式”。

5.5 代码评审：codex review

```
codex review --uncommitted
codex review --base main
```

从本地帮助看，review 适合两类场景：

- 你还没提交，只想先看当前未提交改动的问题；
- 你想把当前分支和某个基线分支做对比审查。

5.6 恢复、分叉、应用结果

```
codex resume --last
codex fork --last
codex apply <TASK_ID>
```

这几个命令很实用：

- resume：接着上次会话干，不用重新讲背景。
- fork：把当前思路分成两条线，试不同方案。
- apply：把某次任务产生的 diff 应到本地工作区。

5.7 两个高频参数

```
codex --search
codex --add-dir ../shared-lib
```

--search 会打开实时网页搜索能力；--add-dir 用来在主工作目录之外，再给 Codex 多一个可写目录。

5.8 什么时候该开 --search，什么时候只看本地代码

`--search` 很有用，但不适合默认一直开着。最稳的判断方法很简单：**只要问题的答案主要藏在当前仓库里，就先别联网；只有当你真的需要外部最新资料时，再把搜索打开。**

更适合**只看本地代码**的情况，通常有这些：

- 你要找入口文件、调用链、核心目录；
- 你要判断当前分支到底改了什么；
- 你要分析仓库里已有的报错、测试、脚本和配置；
- 你要决定“最小改动点”到底落在哪个文件。

更适合**打开 `--search`** 的情况，通常有这些：

- 你要查官方文档、SDK 当前版本行为、参数变化；
- 你要看外部 API、依赖库、平台规则的最新说明；
- 你要确认某个错误码、版本差异、产品限制是不是最近才变过；
- 你明确需要来源链接，而不是只要一句经验判断。

一个很稳的问法，是把本地事实和外部资料拆成两轮：

```
先只基于当前仓库回答：登录失败最相关的文件、调用链和最小改动点是什么？
```

```
再结合官方文档或可信来源，补充说明这里依赖的 SDK 在当前版本里有没有已知限制。
请把“仓库里实际行为”和“外部资料里的说法”分开写，并给出来源。
```

注意

如果你一上来就把联网搜索和本地代码问题混在一起问，很容易得到“看起来很懂，但落不到当前仓库”的回答。先拿到本地事实，再补外部资料，通常更稳。

提示

如果你只记三个入口，先记 `codex`、`codex exec`、`codex review`。这三类已经覆盖了大多数入门场景。

小结

这一章的重点不是全部记住，而是知道怎么分：交互式用 `codex`，脚本化用 `codex exec`，改动复核用 `codex review`。

6

权限、沙箱与审批机制

这一节学完你能做什么

学完这一节，你会明白为什么 Codex 有时能直接跑命令，有时会停下来问你；也会知道不同沙箱模式分别意味着什么。

这部分很重要。很多人第一次用 Codex CLI，会困惑于下面这些现象：

- 为什么它能读文件，但不能写？
- 为什么有些命令会要求你批准？
- 为什么默认情况下网络能力受限？
- 为什么 CI 里和本地交互时的习惯不一样？

答案基本都和**沙箱（sandbox）**和**审批策略（approval policy）**有关。

6.1 先记住三个沙箱级别

模式	大致含义	适合什么场景
`read-only`	只能读，不能改	你只是想看项目、看报错、做分析
`workspace-write`	可以在工作区内读写	本地日常开发，最常用
`danger-full-access`	权限非常大	只在你非常明确风险时才考虑

6.2 再记住三个审批策略

- `untrusted`：只让它自动跑已知安全的读操作，不信任的命令先问你。
- `on-request`：由代理判断什么时候需要向你请求批准。
- `never`：不弹审批，适合自动化场景，但要配合你自己选好的沙箱。

6.3 最常见的组合

官方文档里给了几组很实用的默认组合，可以直接拿来理解：

- 自动但相对稳妥：

```
--full-auto
```

它本质上等于：

```
--sandbox workspace-write --ask-for-approval on-request
```

- 只读浏览：

```
--sandbox read-only --ask-for-approval on-request
```

- 工作区可写，但不信任命令必须先批：

```
--sandbox workspace-write --ask-for-approval untrusted
```

- 完全放开：

```
--dangerously-bypass-approvals-and-sandbox
```

6.4 为什么很多人推荐从 workspace-write 开始

因为它比较符合本地开发的真实需求：

- 能改你当前项目里的文件；
- 不会默认把整个机器都放开；
- 和审批策略配合时，心里更有数。

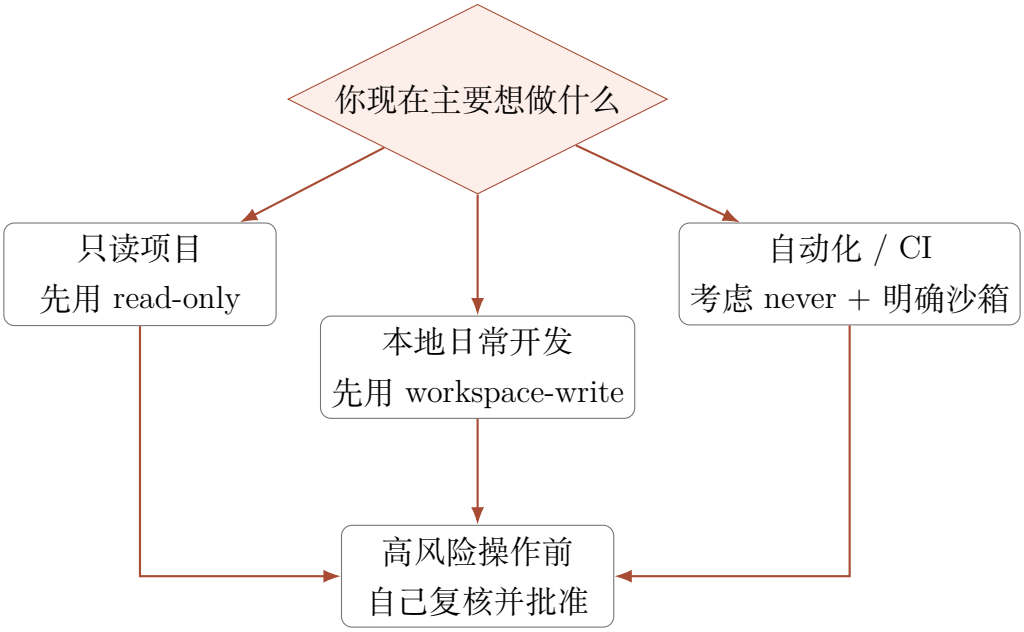
6.5 什么时候会要求你批准

官方文档明确提到，在 `untrusted` 模式下，Codex 会自动运行已知安全的读操作；但**会改变状态、可能有副作用、或会走外部执行路径的命令**，通常都要你批准。比如某些破坏性 Git 操作，就是典型高风险操作。

注意

`--dangerously-bypass-approvals-and-sandbox` 这个名字已经说得很直白了。只有在你外层环境本身就已经被严格隔离时，才考虑用它。

6.6 一个判断思路图



小结

最实用的经验是：本地开发从 `workspace-write` 起步，先别碰完全放开模式；你越清楚权限边界，Codex 用起来就越稳。

7

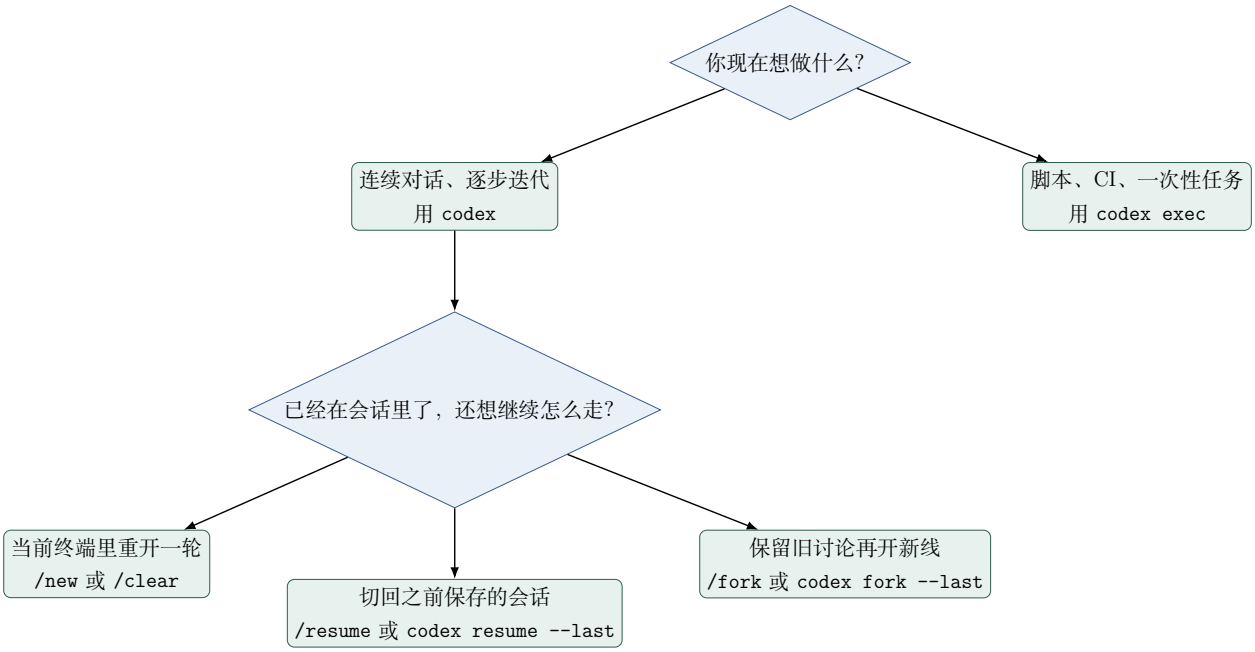
实用斜杠命令与会话控制

这一节学完你能做什么

学完这一节，你会知道在交互会话里有哪些高频斜杠命令，不用退出重进也能切模型、看状态、看 diff、做评审。

CLI 里很多高频操作，不一定要回到命令行里敲完整命令。官方文档的斜杠命令页里，有几条特别适合新手先记住。

7.1 先看这张入口选择图



提示

一句话判断：要连续聊就进 codex；要脚本化就用 codex exec；已经在会话里还想换路，就优先想 /new、/resume、/fork。

7.2 最值得先记的几条

斜杠命令	作用	最常见用途
/status	看当前会话状态	确认模型、审批策略、可写目录、token 使用情况
/model	切模型	当前任务需要更快或更深的推理时切换
/permissions	改权限模式	临时把会话改成更保守或更自动
/plan	先出执行计划	任务较大时先计划再动手
/diff	看 Git diff	改完先审一眼再提交
/review	做工作区评审	查行为风险、缺测试、潜在回归
/new	开一段新会话	想在当前 CLI 里重开一轮，不退出终端
/clear	清屏并新开会话	想把终端内容清掉，再开始新对话
/resume	恢复旧会话	想在交互里切到之前保存的会话
/fork	从旧会话分叉	想保留旧上下文，但另开一条新线
/mention	显式提某个文件	强制把某个路径拉进上下文
/compact	压缩长会话	对话很长时瘦身，保留重点
/init	生成 AGENTS.md 脚手架	把仓库规则写成长期提示

7.3 /status：先确认它到底在什么状态

官方文档对 /status 的描述很实用：它会显示当前模型、审批策略、可写根目录、token 使用情况。也就是说，当你觉得“它怎么突然行为不对劲”时，先跑 /status 通常最省时间。

7.4 /permissions：会话里随时改权限

你一开始可以保守一些，先读代码；真正要改时，再把权限调到更适合当前任务的模式。这样比整场会话从头到尾都开得很大更稳。

7.5 /review 和 /diff：改完一定要连着看

这两条更稳的顺序是连着用：

1. 先用 /review 让 Codex 从行为和测试角度看一下工作区；
2. 再用 /diff 看看具体改了哪些文件、哪些行。

7.6 /plan 和 /init：一个管当前任务，一个管长期规则

/plan 适合当前任务比较大时先做执行计划。/init 则会在当前目录生成 AGENTS.md 脚手架，这个文件可以把“如何跑项目、如何测试、哪些目录别碰、完成标准是什么”这些长期规则写进去。

提示

如果你经常重复说“先跑测试再改”“不要动生成文件”“改完给我验证步骤”，就该考虑用 /init 和 AGENTS.md 固化这些规则了。

7.7 /compact 和 /mention：上下文太长时特别有用

/compact 适合会话已经很长、背景很多，但你还想继续沿着同一个目标聊下去的时候。它的核心作用是把长对话压缩成更短的可继续上下文。
/mention 则适合你想强制把某个文件、目录或路径拉进当前上下文，不希望 Codex 只做泛泛回答的时候。

提示

如果你已经开始反复贴长日志、长代码、长历史结论，就该优先考虑 /compact；如果它总是没有抓到你要看的文件，就用 /mention 点名。

7.8 /model：什么时候值得切模型

/model 适合两种情况：

- 你想先用更快的模型做浏览、定位、整理；
- 你准备进入更复杂的分析、修改或审查，想换到更强的模型。

7.9 /new 和 /clear：在当前终端里重开一轮

/new 就是用来在**当前 CLI 里开启一段新的对话**的，不用先退出终端。
/clear 的作用更直接一些：**先清空当前屏幕显示，再开始一段新的对话**。如果你只是想重新聊一轮，但不在乎清不清屏，用 /new 就够了。

```
/new
/clear
```

7.10 /resume 和 /fork：恢复旧会话，或者从旧会话分叉

/resume 用来恢复之前保存过的会话；/fork 则是基于已有会话分一条新线。对于“我能不能在多个会话间切换”这个问题，最接近的答案是：**可以切到别的已保存会话，但方式是恢复或分叉，不是像浏览器标签页那样常驻并排切**。

```
/resume
/fork
```

如果你在命令行里操作，也可以直接用：

```
codex resume --last
codex fork --last
```

7.11 什么时候退出后重开 codex？

如果你是想连终端态都一起重置，或者你已经退出了交互，那当然也可以直接重新运行：

```
codex
```

但从日常使用角度看，优先级通常是：

- 想原地开启一轮新对话：/new
- 想清屏后再开：/clear
- 想切到旧会话：/resume
- 想从旧会话分一条新线：/fork
- 想从命令行直接恢复最近一次：codex resume --last

7.12 会话命令对照图

/new

在当前终端里直接开一段新的对话。
适合：想重开一轮，但不想退出 Codex。

/clear

先清空当前屏幕，再开始新的对话。
适合：内容太乱，想顺手清屏。

/resume

恢复之前保存过的会话。
适合：切回某个旧会话，继续原来的上下文。

/fork

基于已有会话分一条新线。
适合：保留旧思路，但另开一个方向继续试。

提示

一句话记忆：`/new` 是重开，`/clear` 是清屏重开，`/resume` 是回到旧会话，`/fork` 是从旧会话分叉。

小结

斜杠命令的核心价值，是让你在同一场会话里完成“看状态、改权限、切模型、看 diff、做代码审查”这些高频动作，而不用来回跳出会话。

8

最小可运行示例

这一节学完你能做什么

学完这一节，你至少能照着完成一轮最小体验：进入目录、发起任务、继续追问、复核结果。

这一节只做一件事：帮你把第一轮真的跑通。它和后面的“三轮对话示例”分工不同，前者是最短路径，后者是教你怎么把同一个问题越问越准。

8.1 示例目标

假设你刚进入一个项目目录，想知道：

- 这个项目的入口在哪；
- 哪几个文件最值得先看；
- 如果你是新手，阅读顺序应该是什么。

8.2 一轮最小示例

```
cd your-project
codex
```

进入交互后，你可以这样说：

> 请先查看当前项目结构，告诉我入口文件、核心模块、建议的阅读顺序，并尽量用新手能听懂的话解释。

如果结果不错，继续追问：

> 请进一步解释最核心的两个文件各自负责什么、它们之间怎么配合，以及我如果要改登录逻辑应该先看哪里。

8.3 最后别忘了做一次会话内检查

```
/status
```

看看当前模型、权限模式、可写目录是不是和你预期一致。如果要审改动，再接：

```
/diff
```

注意

如果它只给你空泛总结，没有落到具体文件名、模块名和路径，那就继续追问，不要直接收工。

小结

最小示例的目标不是展示高级能力，而是让你真正建立起“我已经会跑通第一轮”的感觉。

9

三轮对话示例：怎么把问题越问越准

这一节学完你能做什么

学完这一节，你会知道为什么很多好结果不是“第一问神准”，而是靠两三轮追问把范围一点点压出来。

前一节解决的是“能不能顺利跑起来”，这一节解决的是“同一个问题怎么越问越准”。如果你现在只想先把工具跑通，这一节可以先略读，等真正开始多轮迭代时再回来细看。

很多新手第一轮就想拿到最终答案，但真实工作里，更稳的顺序通常是：先拿地图，再抓主线，最后要求可交付结果。下面用“看懂陌生仓库里的登录逻辑”为例。

9.1 第一轮：先把地图拿到手

```
请先按新手视角查看当前仓库，告诉我入口文件、和登录最相关的目录，以及推荐
阅读顺序。
先不要修改代码。
```

这一轮的目标不是立刻修东西，而是先让 Codex 把范围从“整个仓库”收成“和登录相关的那几个入口、目录、文件”。

9.2 第二轮：把范围压到功能级

```
现在只聚焦登录流程。请列出最相关的 3 到 5 个文件，按调用顺序解释它们分别
做什么，
并指出如果我要改“登录失败提示文案”，最可能先看哪一个文件。
```

这一轮你已经不是在听泛泛介绍，而是在逼它回答三件更具体的事：

- 到底是哪些文件最相关；
- 这些文件之间怎么串起来；
- 如果要改一个很小的点，入口最可能在哪。

9.3 第三轮：要求它给可交付结果

```
请只修改登录失败提示文案，不要改业务逻辑，也不要顺手重构。
改完后请按“改动文件 / 为什么改 / 如何验证 / 剩余风险”四项总结。
```

到这一轮，任务边界已经很清楚了：功能只看登录失败提示，改动只碰文案，不做顺手优化，最后还要交付可审的总结格式。

注意

如果第二轮你还没拿到具体文件和调用链，就先别急着进入第三轮。追问不是浪费时间，而是在给后面的修改省返工。

小结

一条很稳的追问节奏是：第一轮拿地图，第二轮抓主线，第三轮要交付。这样通常比“一句话把所有要求塞满”更容易得到靠谱结果。

10

高质量提示词模板库

这一节学完你能做什么

学完这一节，你会拿到一组可以直接复制、稍改就能用的模板，而不是每次临场现编。

很多时候，Codex 用得好不好，不在于你是不是写了“高级提示词”，而在于你有没有说清楚**目标**、**范围**、**约束**、**输出格式**。

10.1 模板一：看懂陌生项目

```
请先从新手视角介绍当前项目。
重点告诉我：
1. 入口文件在哪里
2. 核心目录分别负责什么
3. 主流程怎么走
4. 建议我按什么顺序阅读
请尽量引用具体文件路径，不要泛泛而谈。
```

10.2 模板二：定位某个功能

```
请帮我定位“登录”相关代码。
我想知道：
1. 入口在哪里
2. 涉及哪些文件
3. 调用链大概怎么走
4. 如果我要改登录失败提示，应该先看哪里
```

10.3 模板三：修一个明确的问题

```
我遇到了这个报错，请先不要急着改代码。
请按下面顺序回答：
1. 最可能的原因
2. 建议的排查顺序
```

```
3. 最值得先看的文件
4. 如果要修，最小改动方案是什么
```

10.4 模板四：做一个小改动，但限制范围

```
请只修改提示文案，不要改业务逻辑。
修改前先告诉我你准备改哪些文件。
修改后请总结：
1. 改了哪些文件
2. 每个文件改了什么
3. 我应该怎么验证
```

10.5 模板五：做代码审查

```
请像代码审查者一样检查当前改动。
重点关注：
1. 行为变化
2. 潜在回归
3. 边界情况
4. 缺失的测试
请先给发现的问题，再给简短总结。
```

10.6 模板六：让它先采访你

这是官方最佳实践页面里很值得学的一条思路：如果你的需求还很模糊，可以先让 Codex 反过来问你问题，把模糊目标问清楚。

```
我现在只有一个模糊想法，请先通过提问帮我把需求收紧，挑战我的假设，等目标
清楚以后再开始修改代码。
```

提示

真正高质量的提示词，不是写得花，而是让 Codex 明白“你到底要它做什么、不许它做什么、最后要怎么交付结果”。

小结

写提示词时，尽量补齐四要素：目标、范围、限制、输出格式。这个习惯比背模板更重要。

11

几个典型使用场景

这一节学完你能做什么

学完这一节，你会更容易判断什么时候值得用 Codex CLI，什么时候自己手动做反而更快。

这一节更像场景索引，不是新的主线章节。你不一定要逐段精读；更适合在真正碰到“接手项目”“处理报错”“做代码审查”这些任务时，回来按场景抄一句起手式。

11.1 场景一：接手陌生项目

你最需要的不是“全量介绍”，而是“给我一个阅读入口”。

```
请按新手视角介绍这个项目，重点说入口、核心目录、主要数据流，并给出推荐阅读顺序。
```

11.2 场景二：定位某个功能

```
请帮我定位上传功能相关代码，告诉我涉及哪些文件、入口在哪里、调用链大概是什么。
```

11.3 场景三：解释报错

```
这是我刚遇到的报错。请先解释最可能的原因，再给我一个排查顺序，最后告诉我应该先看哪些文件。
```

11.4 场景四：改一个小功能

```
请把这里的提示文案改得更清楚，但不要改业务逻辑。修改后请说明改了哪些文件，以及如何验证。
```

11.5 场景五：提交前做一次代码审查

```
/review
/diff
```

这是很典型的一套动作：先让它帮你找风险，再自己看差异细节。

小结

Codex CLI 最值钱的地方，往往不是帮你“一口气写完整个功能”，而是大幅缩短“理解、定位、修改、复核”这几步之间的时间。

12

和 Git、测试、非交互模式的配合

这一节学完你能做什么

学完这一节，你会知道如何把 Codex CLI 接进真实开发流程，而不是只停留在聊天式体验。

12.1 和 Git 的最小配合方式

你可以把一轮工作想成这样：

1. 先让 Codex 解释或修改；
2. 再让它做 `/review` 或 `codex review`，先把行为风险和缺失测试圈出来；
3. 然后用 `/diff` 或 Git diff 看差异，确认具体改了哪些文件、哪些行；
4. 最后自己跑测试、决定是否提交。

如果你想用命令行方式做审查，本地帮助里有这些入口：

```
codex review --uncommitted
codex review --base main
```

12.2 改坏了怎么办：先保留现场，再决定怎么撤

新手最需要补的一课，其实不是“怎么让它改”，而是“改得不对时怎么稳稳收回来”。最怕的不是改坏，而是你在没看清现场时又继续乱改。

更稳的顺序通常是：

1. 先看 `git status`；
2. 再看 `git diff`；
3. 如果当前现场还值得保留，先切一条救援分支，或者先做一次 `stash`；
4. 最后才决定是继续修、部分撤回，还是整批撤回。

```
git status
git diff
git switch -c backup/codex-rollback-check
git stash push -u -m "codex-before-rollback"
```

如果你只是想撤掉某一个文件的工作区改动，可以先从最小动作开始：

```
git restore path/to/file
```

如果你非常确定这批跟踪文件里的改动都不要了，才考虑：

```
git restore .
```

这时也可以先让 Codex 帮你“做判断”，而不是“直接执行”：

```
先不要直接撤回任何东西。请先根据当前 diff 告诉我：
1. 哪些改动明显是这次任务引入的
2. 哪些文件值得保留
3. 如果我要最小回滚，应该先撤哪几个文件
4. 每一步回滚后该怎么验证
```

提示

回滚不等于“一键清空”。先保留证据，再缩小范围，再撤最小那一块，通常比一上来整批回退更稳。

注意

别把 `git reset --hard` 这类命令当默认答案。只要你还没想清楚哪些内容要保留，就先别碰这类大锤。

12.3 改完以后，别只看它说了什么

真正重要的是验证：

- 它改的文件是不是你预期的文件；
- 它有没有顺手碰到不该碰的模块；
- 构建、测试、关键路径是不是还过得去。

12.4 如果你知道测试命令，最好明确告诉它

官方最佳实践里有一个很关键的提醒：如果你不告诉 Codex 该怎么构建、怎么测试，它就不容易验证自己的工作。所以你可以在提示里直说：

这里顺手解释一下：`lint` 一般指**静态检查**，主要用来提前发现格式、风格和一些明显的代码问题。

> 改完以后请运行项目已有的测试命令。如果你不确定，先搜索仓库里现有的测试、构建和静态检查脚本再决定。

12.5 把长期规则写进 AGENTS.md

官方最佳实践明确建议，把重复出现的规则沉淀到 `AGENTS.md`，而不是每次都重新说一遍。一个好用的 `AGENTS.md` 往往会包括：

- 仓库结构和重点目录；
- 构建、测试、静态检查命令；
- 工程约定和禁止事项；
- “算完成”的标准和验证方式。

你可以先用：

```
/init
```

生成脚手架，再按团队真实习惯补充内容。

12.6 除了 AGENTS.md，也值得把常用设置固化进 config.toml

`AGENTS.md` 管的是**仓库规则**；`~/.codex/config.toml` 更适合管你自己的**默认工作习惯**，比如常用模型、审批策略、沙箱模式，以及几种固定的 profile。

如果你每次进会话都在重复同一种组合，通常就值得把它固化下来。一个很实用的分法是：

- **safe**：只读浏览，适合看项目、看报错、做方案比较；
- **dev**：工作区可写，适合本地日常开发；
- **local**：走本地开源模型，适合做低风险浏览、试验或离线场景。

日常使用时，你真正需要记住的，常常只是这些入口：

```
codex -p safe
codex -p dev
codex --oss --local-provider ollama -p local
```

如果只是偶尔临时换一次，也不一定要改配置文件，直接在命令行里覆盖就够了：

```
codex -s read-only -a on-request
codex -s workspace-write -a on-request --search
```

提示

最值得固化的，不是“所有参数”，而是你一周里会重复用很多次的那两三种工作模式。

注意

配置文件字段名和可用项可能会随版本演进变化。真正落地时，还是以你本地帮助和当前官方配置文档为准。先想清楚“我要固化哪几种模式”，再去写具体字段，通常更不容易乱。

12.7 非交互执行：什么时候该用 codex exec

如果你想把某些工作做成脚本，或者在 CI 里跑，codex exec 很适合。比如：

```
codex exec "总结当前仓库最重要的三个风险点"
```

它还支持把最终消息写到文件里：

```
codex exec "生成本次改动总结" -o summary.txt
```

如果你要方便程序读取的输出，可以加：

```
--json
```

如果你要严格结构化结果，可以再加：

```
--output-schema
```

12.8 CI 里怎么认证

官方非交互文档明确建议：在 CI 里优先用 API 密钥，并通过 CODEX_API_KEY 注入。它还特别指出：CODEX_API_KEY 这个环境变量**只在 codex exec 里支持**。

```
CODEX_API_KEY=<api-key> codex exec --json "梳理当前未处理的问题报告"
```

12.9 把 exec 真正接进脚本：人看摘要，机器看结构

很多人把 codex exec 用到“命令能跑通”就停了，但脚本闭环通常还差一步。更稳的自动化思路，是把输出拆成三层：

- 给**人**看的最后摘要；
- 给**脚本**读的结构化事件流；
- 给**流程**判断的退出状态和文件存在性检查。

一个很实用的最小样板可以长这样：

```
set -euo pipefail
export CODEX_API_KEY=your_api_key

codex exec --json \
  --output-schema review-schema.json \
  -o codex-summary.txt \
  "审查当前改动，并按约定结构输出风险摘要" \
  > codex-events.jsonl
```

这几个产物各有分工：

- `codex-summary.txt`：给人看，一眼知道结论；
- `codex-events.jsonl`：给脚本留结构化事件流，后面要上传日志、做二次处理会方便很多；
- `review-schema.json`：约束最终输出格式，让下游流程更稳定。

跑完后，脚本里至少再补一个很小的检查：

```
test -s codex-summary.txt
```

这样做的意义很直接：就算这轮任务没有正常产出摘要，流水线也能尽早失败，而不是把“空结果”继续往后传。

提示

如果后面真的要自动消费结果，尽量让机器读 `--json` 或 `--output-schema` 约束过的输出，不要去硬拆自然语言段落。

12.10 为什么 exec 默认要求 Git 仓库

官方文档也写得很清楚：`codex exec` 默认要求在 Git 仓库内运行，这是为了降低破坏性改动风险。如果你非常确定环境安全，才考虑：

```
codex exec --skip-git-repo-check "..."
```

注意

把一项任务做成自动化之前，先手工把它跑稳定。官方最佳实践也明确不建议过早把不稳定流程自动化。

小结

从“能聊天”到“能进工作流”，关键就三件事：看 diff、跑测试、把重复规则沉淀到 `AGENTS.md`。而脚本化场景则优先考虑 `codex exec`。

13

任务完成验收卡

这一节学完你能做什么

学完这一节，你会知道一轮任务做到哪一步才算真正收工，而不是“它说改完了，我就先信了”。

很多人不是不会用 Codex，而是不知道什么时候该停。最稳的做法，是在结束前拿下面这张验收卡过一遍。

检查项	最低标准	不够清楚时可以继续怎么问
改动范围	只碰到预期文件，没顺手改一堆无关内容	请列出本轮改动文件，并说明每个文件为什么必须改
验证情况	至少说明跑了哪些检查，或者明确哪些没跑	请只列出你实际执行过的验证，以及还没验证的部分
结果说明	能用通俗的话讲清楚“这次到底改了什么”	请用业务视角总结本次变化，不要只重复 diff
剩余风险	能指出最值得我再看的边界和潜在回归点	请列出当前改动最值得我复核的两个风险点
提交准备	你自己能判断现在适不适合提交或继续补检查	基于当前状态，你建议现在提交，还是先补哪一步

如果你想让 Codex 按这张卡汇报，可以直接说：

```
请按“改动文件 / 验证情况 / 剩余风险 / 是否建议现在提交”四项总结这次工作。
```

注意

不要把“它说自己验证过了”直接当验收结果。你真正要看的，是它到底验证了什么、没验证什么，以及这些检查够不够覆盖你的关键路径。

小结

任务完成的判断标准不是“它停下来了”，而是你已经看清改了什么、怎么验的、还剩什么风险，以及现在能不能安心提交。

14

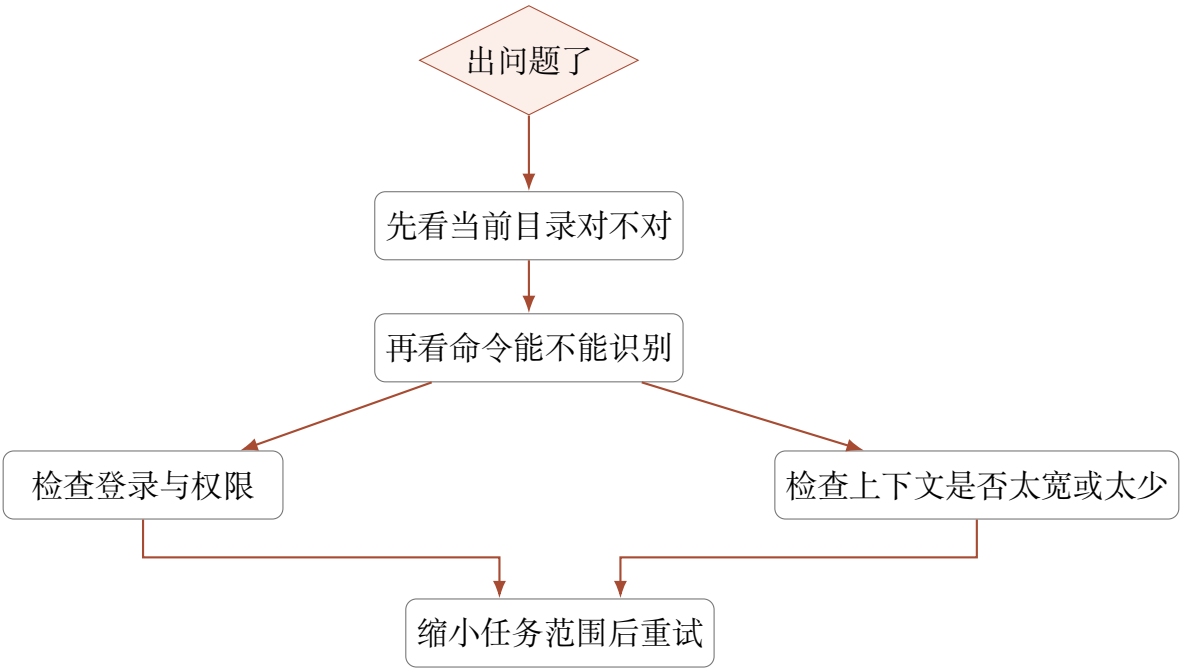
常见错误与排查思路

这一节学完你能做什么

学完这一节，你遇到问题时至少知道先查什么，不会一报错就完全没方向。

最常见的问题，通常不是模型不聪明，而是环境、目录、登录、权限、上下文这几类基础问题。

14.1 排查思路图



14.2 问题一：命令找不到

```
command not found: codex
```

优先检查：

- 是否真的安装成功；
- 当前命令行环境能不能找到 `codex`；

- 终端会话是否需要重新打开。

14.3 问题二：明明在项目里，却读不到对的内容

先确认你是不是在正确目录：

```
pwd
ls
```

如果目录没问题，再看你是不是把任务说得太宽了。

14.4 问题三：回答太空泛

大概率不是它完全不会，而是你的任务太宽。解决办法通常是：

- 缩小目标；
- 指定文件、模块或功能；
- 明确输出格式，比如“列出文件路径 + 每个文件一句解释”。

14.5 问题四：它总在问你要不要批准

这不一定是坏事。多数时候只是当前审批策略较保守，或者它准备执行的命令本来就不属于安全读操作。

你可以先用：

```
/status
/permissions
```

看当前会话权限，再决定要不要调模式。

14.6 问题五：脚本里跑不通

如果你在 CI 或脚本里用 `codex exec`，优先确认：

- 认证是不是用的 `CODEX_API_KEY`；
- 当前目录是不是 Git 仓库；
- 是否需要 `--output-last-message` 或 `--json` 这类更适合自动化的输出方式。

注意

高风险操作、删除操作、覆盖性修改，一定要自己复核。AI 可以帮你提速，但不能替你承担最终判断责任。

小结

排查时优先看五件事：目录、安装、登录、权限、任务范围。先把这五件事排清楚，通常已经能解决大部分问题。

15

使用建议与最佳实践

这一节学完你能做什么

学完这一节，你会知道怎样把 Codex CLI 用得更稳、更省时间，而不是越用越乱。

15.1 建议一：任务要具体

尽量少说“帮我优化一下”，多说“帮我解释这个目录”“帮我定位登录逻辑”“只改文案，不改业务逻辑”。

15.2 建议二：先小后大

别一上来就扔超大任务。更稳妥的顺序通常是：

1. 先解释；
2. 再定位；
3. 再计划；
4. 最后再修改。

15.3 建议三：让它输出结构化结果

比如直接要求：

- 按文件列表输出；
- 每个文件一句话解释；
- 先结论，再细节；
- 最后给验证建议。

15.4 建议四：让它知道怎么验证

官方最佳实践明确提醒，不要让代理在不知道怎么构建、测试、做静态检查的前提下瞎试。所以最好在仓库里维护好 `AGENTS.md`，或者在提示中说明验证方式。

15.5 建议五：复杂任务先计划

当任务比较长、改动面比较广时，先用 `/plan` 或者直接让它先出计划，再决定是否实施。这样能明显减少“上来就改，结果越改越散”的问题。

15.6 建议六：把重复规则沉淀下来

如果你经常重复同样的话，就说明这条规则应该搬进 `AGENTS.md`、技能文件或配置，而不是每次重新输入。

提示

把 Codex CLI 当成“会先打草稿、再陪你迭代”的工程搭档，比把它当成“全自动代打工具”稳定得多。

小结

最佳实践就一句话：任务说清楚，范围控住，验证补上，重复规则沉淀下来。

16

新手最容易踩的坑

这一节学完你能做什么

学完这一节，你会提前知道哪些地方最容易翻车，尤其适合第一次正式使用前快速扫一遍。

1. 没确认当前目录，就开始让它读代码或改文件。
2. 任务描述太宽，结果很空，还误以为是工具不行。
3. 一上来就给太大权限，自己却没想清楚风险。
4. 看见它改完了就直接接受，没有自己看 diff 和跑验证。
5. 把“解释、定位、修改、验证”四件事全揉进一个超长需求里。
6. 在自动化场景里直接照搬交互式习惯，没有单独处理认证和输出。
7. 有重复规则却不写进 `AGENTS.md`，导致每次都要重新解释。

注意

新手最大的问题通常不是不会敲命令，而是没有形成稳定节奏：先确认，再限制范围，再验证结果。

小结

避坑的关键不是记住一堆例外，而是养成流程：确认目录、收紧任务、看状态、审改动、做验证。

17

高风险任务边界清单

这一节学完你能做什么

学完这一节，你会更清楚哪些任务不适合一上来就让 Codex 放手执行，尤其是在你还没做人工复核的时候。

高风险任务不是“完全不能让 Codex 参与”，而是**不能直接把执行权整包交出去**。下面这些情况，尤其值得先收紧范围、先看计划、先做人工确认。

这类任务为什么高风险	更稳的做法
删除、覆盖、批量改名、批量格式化：一旦范围没控住，回滚和代码审查都会很痛苦	先要求它只列计划和影响范围，再按小批次执行，改完立刻看 `/diff`
破坏性 Git 操作：比如重写历史、强推、回退到旧提交	先自己决定策略，再让它辅助解释影响，不要把最终判断外包出去
数据库迁移、数据回填、状态修复脚本：副作用大，出错后代价高	先让它写风险说明、回滚思路和验证步骤，再由你确认后执行
密钥、认证缓存、生产配置：一旦泄漏或改错，后果往往比代码问题更严重	先把敏感文件和目录说清楚，必要时明确要求“只解释，不修改”
会触发外部副作用的命令：部署、发请求、写远端资源、修改系统状态	优先用只读方式分析，真正执行前一定要自己审一遍命令含义和影响范围

如果你已经知道这是高风险任务，一个很好用的开场方式是：

```
这是一个高风险任务。先不要执行，也不要改文件。
请先列出影响范围、主要风险、回滚方法，以及你建议我先看的最小变更范围。
```

提示

高风险任务最怕的不是“工具不够聪明”，而是你没有先把边界、回滚和验证想清楚。先让它帮你分析，再决定要不要放行，通常更稳。

小结

判断一件事要不要谨慎放手，有个很简单的标准：如果做错之后难回滚、难做代码审查、代价高，那它就属于高风险任务。

18

进阶使用技巧：从会用到会控场

这一节学完你能做什么

学完这一节，你会开始把 Codex CLI 当成真正的工程搭档来用，不只是“帮我改一段代码”，而是会主动控范围、控节奏、控交付。

前面那些内容已经够你顺利上手了。到了进阶阶段，重点就不再是“知道有哪些命令”，而是**知道什么时候该先想、什么时候该先收范围、什么时候该先比方案、什么时候该固定输出格式**。

18.1 技巧一：先做方案评审，不急着改代码

适合什么时候：需求还没完全定、同一个目标可能有两三种实现路径、你担心改动面会扩散的时候。

核心思路：先让 Codex 做“方案比较”，不是立刻进入“执行模式”。

```
先不要改代码。请给我 2 到 3 种实现方案，分别说明：
1. 需要改哪些文件
2. 改动范围大概多大
3. 主要优点和风险
4. 你更推荐哪一种，为什么
```

这类问法很适合放在需求刚成形的时候。它能帮你更早看到“最小改法”和“最顺手但改动更大”的区别。

注意

常见误用是：嘴上说“先给方案”，后面又补一句“顺手直接改掉”。这样很容易让会话直接滑进执行阶段，失去先比较的价值。

18.2 技巧二：大仓库先控上下文，不要一口气全拉进来

适合什么时候：仓库很大、目录很多、你已经开始感觉回答越来越空，或者会话里反复塞长日志、长代码的时候。

核心思路：先收文件范围，再收功能范围，最后再收动作范围。

```
先不要全仓库泛讲。请先找出和“登录失败提示”最相关的 5 个文件，
只输出文件路径，并说明我下一轮应该重点追哪两个。
```

这一招的关键，不是让它“知道更多”，而是让它**只盯真正相关的那一小块**。配合 /mention 和 /compact，通常会比不断贴更多背景更有效。

提示

大仓库里一个很稳的顺序是：先问相关文件，再问调用链，再问最小改动点。先别一上来就要求“完整分析整个系统”。

18.3 技巧三：用 /fork 保留旧思路，同时试另一条线

适合什么时候：你已经积累了一轮上下文，但想试另一种问题假设、另一种实现方案、或者另一种排查顺序。
核心思路：不要推翻当前会话，而是把它当成一条已经建立好的分支，再开一条线比较。

```
/fork
```

你可以在分叉后的会话里直接说：

```
保留上一轮的上下文，但这次请不要沿用原来的结论。
请从“问题可能出在缓存或状态同步”这个角度重新给排查顺序。
```

/fork 最值钱的地方，是它能让你把“旧结论”和“新猜想”并排保留，而不是每次都从零重讲一遍。

注意

常见误用是把 /fork 当“重开会话”。如果你只是想清掉当前聊天，不需要保留对比关系，那应该优先用 /new 或 /clear。

18.4 技巧四：用 /model 做快慢分工

适合什么时候：你想先快速扫目录、列文件，再进入更复杂的分析、修改或代码审查。
核心思路：把“浏览定位”和“深入执行”拆开，不要整个任务都用同一种节奏。

```
/model
```

一个很稳的工作节奏通常是：

1. 先用更快的模型做扫目录、找入口、列文件；
2. 再切到更强的模型做复杂推理、修改、代码审查；

3. 如果后面又回到大量浏览工作，再切回更快的模型。

这样做的价值不是“追求参数感”，而是让会话节奏更像真实工程工作：先摸清，再深挖。

18.5 技巧五：先定义成功标准，再开始动手

适合什么时候：任务边界模糊、你担心它顺手改太多、或者你自己也还没想清“做到什么算完成”。
核心思路：先把验收条件写出来，再让 Codex 开始做。

```
先不要修改。请先复述目标、列出不做的事、给出完成标准和验证方法。
等我确认后再开始。
```

这类写法特别像你在团队里先对齐需求。它能明显减少“它做了不少事，但不是我真正要的那件事”这种偏差。

提示

如果你发现自己总在说“不是这个意思”“这个也别改”“这个先不做”，那就说明成功标准应该前置，而不是等它做完再修正。

18.6 技巧六：把代码审查做成闭环，不是只跑一次

适合什么时候：你已经改完一轮，准备决定是否提交，或者你怀疑当前修改里还有行为风险。
核心思路：代码审查不是一句结束语，而是一轮小循环。

一个很实用的闭环是：

1. 先修改；
2. 跑一次 `/review`；
3. 只优先修真正高风险问题；
4. 再看 `/diff`；
5. 最后做一轮简短复核。

如果你想让它聚焦真正的问题，可以直接说：

```
请只列真正值得我在提交前处理的问题。
把“高风险问题”和“可选优化”分开写，先说高风险问题。
```

注意

代码审查最常见的误用，是把“它给了一堆建议”当成“都得改”。真正应该先处理的，是那些会引起行为回归、边界错误、缺失验证的问题。

18.7 技巧七：固定结构化交付格式

适合什么时候：你希望每轮结果都更像真实交付，而不是一大段自由发挥。
核心思路：不要每次临场听它发挥，直接固定输出模板。

```
请固定按下面格式汇报：
1. 结论
2. 改动文件
3. 验证情况
4. 剩余风险
5. 下一步建议
```

这招最适合和 AGENTS.md 一起用。因为一旦你发现团队每次都想看同一种结果格式，那它就应该从“临时提示词”升级成“仓库规则”。

提示

结构化交付不是为了好看，而是为了让你更快判断：这轮到底改了什么、验了什么、还差什么、现在能不能交。

18.8 技巧八：把 codex exec 用在真正适合自动化的地方

适合什么时候：你要做批量总结、风险归纳、分类整理、CI 检查、规范化输出，而不是大范围自动改代码。
核心思路：优先自动化“读和总结”，谨慎自动化“改和执行”。

```
codex exec --json "总结当前分支相对 main 的三个主要风险"

codex exec --json \
  --output-schema schema.json \
  "把当前改动整理成结构化交付摘要"
```

进阶阶段，exec 最值得做的通常是这些事：

- 生成结构化改动总结；
- 对当前 diff 做风险分类；
- 统一整理 issue、日志、报错；

- 在 CI 里做只读检查和摘要输出。

而下面这些事，通常不适合太早自动化：

- 还没在本地跑稳的大流程；
- 改动范围大的自动修复；
- 带外部副作用的执行动作；
- 出错后很难回滚的操作。

18.9 技巧九：先用一个小例子理解 MCP 到底值在哪

适合什么时候：仓库外还有产品文档、内部知识库、工单系统、接口说明，或者别的外部工具数据，而你不想在提示词里手动复制来复制去。
核心思路：让 Codex 同时看到“本地代码”和“外部上下文”，但第一次一定先从**一个只读小场景**跑通。

先用命令确认你现在接了哪些 MCP 服务：

```
codex mcp list
codex mcp add docs --url https://example.com/mcp
```

然后拿一个特别具体的任务试水。比如当前仓库里有退款逻辑，外部文档里写着退款接口规范，你就可以这样问：

```
请结合当前仓库里的退款实现，以及 docs MCP 里的“退款接口”文档，
分开告诉我：
1. 本地代码现在实际怎么做
2. 文档要求是什么
3. 两边有没有不一致
4. 如果我要先验证一处，最该从哪开始
```

这类场景里，MCP 的价值就会很清楚：

- 只看本地代码，你知道“现在做成了什么样”；
- 只看文档，你知道“理论上应该怎么做”；
- 两边一起看，你才能更快找出偏差点和验证顺序。

第一次接 MCP，最值得先试的通常不是“让它直接调用很多外部系统”，而是下面这类只读任务：

- 本地实现和外部文档是否一致；

- 仓库里的字段命名和接口定义是否对得上；
- 工单里描述的问题，能不能在代码里快速对上位置。

提示

第一次接 MCP，不要一上来把工单、监控、文档、数据库全接上。先接一个只读来源，先做“对照、解释、定位”，跑顺后再往外扩。

注意

MCP 让上下文更丰富，不等于判断可以外包。特别是带写入能力的外部服务，依旧要沿用前面讲的审批、范围和回滚思路。

小结

进阶使用的核心，不是会更多命令，而是会控四件事：控范围、控节奏、控比较、控交付。你一旦开始主动控制这四件事，Codex 的稳定性会明显上一个台阶。

19

正文收束：3 分钟速读总结

到这里为止，正文主线已经讲完了。下面先用一页把前面的重点收口；再往后是速查和附录，适合按需跳读，不必再按顺序硬看。

如果你现在只想记住最重要的几点

1. 把 Codex CLI 当成终端里的代码搭档，而不是全自动代打工具。
2. 安装后先做三件事：`codex --help`、登录、最小任务验证。
3. 日常开发多用 `codex`，脚本化多用 `codex exec`；改完更稳的顺序是先 `/review`，再 `/diff`，最后自己验证。
4. 本地开发优先从 `workspace-write` 这类中间档权限开始，不要急着全放开。
5. 高质量提示词的关键是：目标、范围、限制、输出格式。
6. 真正让 Codex 稳定好用的，不是一次神 prompt，而是 `AGENTS.md`、清晰测试方式和稳定工作流。

20

常用命令速查表

这一页不讲新知识，只把正文里出现过的高频入口收成一张表，方便你回头查。看完这页，下一页再配合“新手避坑清单”一起用，会更像一套随手能翻的工具卡。

命令/操作	用途	什么时候用
codex	进入交互式会话	日常本地开发、连续提问、逐步迭代时最常用
codex --help	查看帮助信息	忘了入口、参数、功能时先用
codex --version	查看版本	怀疑版本差异时先确认
codex login	发起登录	想显式登录而不是等首次运行触发
codex login status	查看登录状态	不确定当前是否已认证时
codex exec "..."	非交互执行	脚本化、CI、一次性任务
codex review --uncommitted	审查当前未提交改动	提交前快速做风险检查
codex review --base main	对比分支基线做评审	提交拉取请求（PR）前自查很实用
codex resume --last	恢复最近一次会话	不想重复讲背景时
codex apply <TASK_ID>	应用某次任务产出的 diff	你想把结果落到本地工作树时
/status	查看会话状态	确认模型、权限、可写目录
/permissions	调整权限模式	临时改成更保守或更自动
/diff	查看当前改动	改完先看一眼再提交
/review	在会话内做代码审查	想先查风险和缺失测试
/plan	先做执行计划	任务较大、较复杂时
/init	生成 AGENTS.md 脚手架	想把规则沉淀到仓库里

21

新手避坑清单

这一页和上一页速查表最好搭着看：上一页解决“现在该用哪个入口”，这一页解决“哪些地方最容易因为节奏不稳而翻车”。

- 操作前先确认你在哪个目录。
- 登录方式和使用场景要区分：交互式和 CI 不是一回事。
- 能先解释就先解释，别急着大改。
- 让它改文件时，要求说明改了哪些文件、为什么改、怎么验证。
- 会话里看不对劲，先用 `/status` 和 `/permissions`。
- 改完更稳的顺序是先 `/review`，再看 `/diff`，最后自己验证。
- 有重复规则就写进 `AGENTS.md`，别每次重新说。
- 遇到版本差异时，以你本地 `codex --help` 和官方文档为准。

22

附录：从零到一演练页

这一节学完你能做什么

学完这一节，你会看到一条从克隆公开小仓库到完成一次解释、定位、修改、复核的完整路径，适合第一次照着做。

从这里开始进入附录区。前面的正文已经够你上手；后面的内容更偏演练、模板和查表。

这一页的重点不是仓库本身有多复杂，而是帮你把**真实工作流顺一遍**。它和前面的“最小可运行示例”也不重复：前者是最短体验，这一页是完整演练。这里用“任意公开小仓库”来演示，你可以换成一个最小待办应用、博客或练习项目。

22.1 目标

假设你的目标很简单：

- 先看懂仓库结构；
- 再找到入口；
- 再做一个低风险改动；
- 最后复核并总结。

22.2 第 1 步：把公开仓库拉到本地

```
git clone <公开小仓库地址> demo-repo
cd demo-repo
```

这里故意不把演示绑死到某一个具体仓库名上，是为了让你能把这页直接复用到任何公开小项目。

22.3 第 2 步：第一次进入会话

```
codex
```

第一轮建议先这样问：

```
请先按新手视角查看当前项目。
告诉我：
1. 入口文件在哪里
2. 核心目录分别负责什么
3. 我应该按什么顺序阅读
请尽量引用具体文件路径。
```

22.4 第 3 步：定位一个具体点

等你拿到目录概览以后，继续把范围收紧：

```
现在请重点解释最核心的两个文件。我想知道它们分别负责什么，以及它们之间是怎么配合的。
```

22.5 第 4 步：做一个低风险改动

第一次练手，不要先碰大逻辑，优先选这类改动：

- README 说明补充；
- 文案更清楚；
- 一个明显的小问题；
- 缺失但容易补的测试。

比如你可以这样说：

```
请只做一个低风险改动：把首页提示文案改得更清楚，不要改业务逻辑。修改前先告诉我你准备改哪些文件。
```

22.6 第 5 步：复核改动

改完以后不要立刻收工，先看：

```
/review
/diff
```

再让它补一句人话总结：

```
请总结这次改动：
1. 改了哪些文件
2. 每个文件改了什么
3. 我现在应该怎么验证
```

22.7 第 6 步：完成一轮最小闭环

这时你应该已经完成了最重要的一轮：

1. 进入仓库；
2. 读懂结构；
3. 定位关键点；
4. 做一处低风险修改；
5. 看 diff 和代码审查结果；
6. 准备验证。

提示

第一次练手时，不追求改得多，追求完整走完一轮。能稳定走完这一轮，比一次改十个文件更重要。

小结

“从零到一演练页”的核心不是仓库名，而是让你掌握一条固定路径：先理解，再定位，再小改，再复核。

23

附录：练习题怎么看算合格

这一节学完你能做什么

学完这一节，你会更容易判断自己的练习结果算不算合格。这里不给“标准答案”，而是给“合格答案的样子”。

因为每个仓库都不一样，所以这页不提供唯一答案。更有用的做法，是看你的结果有没有落到下面这些关键点。
这页故意放在“练习题页”前面，是因为你先知道判分标准，后面练的时候会更有尺子；如果你已经做完，也可以回来对照自检。

23.1 练习一：合格结果至少要有路径和顺序

合格结果通常至少要满足三件事：

- 指出具体文件路径，而不是只说“入口大概在前端目录”；
- 说明为什么从这个文件开始看；
- 给出一个能真正照着走的阅读顺序。

```
示例格式：
1. 入口文件：`src/main.ts`
2. 最值得先看的目录：`src/router/`、`src/pages/`
3. 推荐阅读顺序：
   - 先看 `src/main.ts`，确认应用怎么启动
   - 再看 `src/router/index.ts`，确认页面入口怎么串起来
   - 最后看 `src/pages/Login.vue`，把登录页逻辑补上
```

23.2 练习二：合格结果要讲清调用链，不只是列文件

如果你做的是“定位功能”练习，合格结果通常至少要满足三件事：

- 能指出入口或触发点，而不是只堆文件名；

- 能说清最相关的 3 到 5 个文件怎么串起来；
- 能指出如果下一步只改一个小点，最可能先看哪里。

```
示例格式:
1. 入口: `src/pages/Login.vue` 里的提交按钮触发登录动作
2. 关键文件:
   - `src/pages/Login.vue`: 收集输入并触发表单提交
   - `src/store/auth.ts`: 发起登录请求并更新登录状态
   - `src/api/auth.ts`: 封装登录接口
3. 调用顺序:
   页面提交 -> store action -> API 请求 -> 成功后更新状态
4. 如果只改“登录失败提示文案”，先看 `src/pages/Login.vue`
```

23.3 练习三：合格结果要控住范围，还要说清怎么验

如果你做的是低风险小改动，合格结果通常会有这些特征：

- 只改一两个真正相关的文件；
- 明确说出“没有改业务逻辑”或者“没有顺手重构”；
- 给出至少一条可执行的验证方法。

```
示例格式:
- 改动文件: `src/pages/Login.vue`
- 改动原因: 把登录失败提示改得更清楚
- 未改动部分: 没有修改请求逻辑、校验逻辑、路由跳转
- 验证方法: 输入错误密码，确认页面提示文案符合预期
```

23.4 练习四和练习五：重点看它有没有真的“落地”

Review 练习合不合格，重点看这几件事：

- 它有没有优先指出真实风险，而不是只夸；
- 它的评论有没有对上当前 diff；
- 它有没有把“必须改”和“可选优化”分开。

提示词练习合不合格，重点看这几件事：

- 改写后的提示词有没有明确目标、范围、限制、输出格式；
- 结果里是不是明显多了具体文件、动作和验证；
- 你是不是更容易继续追问，而不是继续被迫重开题。

提示

自检时有个很实用的判断法：如果你的结果里能直接看到“路径、动作、限制、验证”，通常就已经比“空泛介绍型答案”强很多了。

小结

练习题最怕的不是答错，而是答得看起来很多，却没有路径、没有边界、没有验证。只要你能把这三样做出来，练习就已经很值了。

24

附录：好提示词 vs 坏提示词

这一节学完你能做什么

学完这一节，你会更容易判断什么样的提问会让 Codex 输出靠谱结果，什么样的提问容易把它带偏。

场景	不太好的写法	更好的写法
看项目	帮我看看这个项目	请按新手视角介绍当前项目，重点说入口、核心目录、主流程和推荐阅读顺序
定位功能	帮我找登录	请帮我定位登录相关代码，告诉我入口、涉及文件和大致调用链
处理报错	帮我修一下这个错	这是报错信息。请先解释最可能原因，再给排查顺序，最后给最小修改方案
改代码	帮我优化一下	请只修改提示文案，不要改业务逻辑，修改后列出改动文件和验证方法
做代码审查	帮我看看有没有问题	请像代码审查者一样检查当前改动，先列问题，再给简短总结，重点关注回归和缺失测试

24.1 一个简单判断法

如果一条提示词同时满足下面四件事，它通常就已经比平均水平强很多：

- 说清楚目标；
- 说清楚范围；
- 说清楚限制；

- 说清楚输出格式。

24.2 最常见的坏味道

- 只有动作，没有目标，比如“优化一下”“改一下”。
- 只有目标，没有范围，比如“把这个项目整理好”。
- 只有结果要求，没有约束，比如“修好它”，却没说能不能改架构。
- 没要求输出格式，导致结果全是大段空话。

小结

真正的区别不在“高级词藻”，而在有没有把目标、范围、限制、输出格式交代清楚。

25

附录：常见场景小抄

这一节学完你能做什么

学完这一节，你可以把常见场景当成小抄来用，遇到任务时直接照着改，不用从头想。

25.1 场景一：接手项目

提示

先问什么：入口、核心目录、主流程、阅读顺序。
推荐提示词：请按新手视角介绍当前项目，引用具体文件路径。
别忘了：先让它解释，不要一上来就改。

25.2 场景二：处理报错

提示

先问什么：最可能原因、排查顺序、最值得看的文件。
推荐提示词：先解释报错，不要急着修改。
别忘了：让它给最小修改方案，而不是上来大改。

25.3 场景三：补测试

提示

先问什么：项目现有测试框架、最接近的已有测试、这次应该补哪种测试。
推荐提示词：请先找项目里现有测试模式，再补一条风格一致的测试。
别忘了：明确要求不要顺手重构无关代码。

25.4 场景四：做代码审查

提示

先问什么：行为风险、边界情况、缺失测试、可能回归。
推荐动作：先 `/review`，再 `/diff`。
别忘了：要求“先列问题，再总结”，这样输出更适合做代码审查。

小结

常见场景小抄的核心作用，是帮你减少临场组织语言的成本。越常见的任务，越值得模板化。

26

附录：CI 集成示例

这一节学完你能做什么

学完这一节，你会知道怎样把 codex exec 接进自动化流程，尤其是怎样组合 --json、-o 和 API 密钥。

26.1 最小命令行示例

```
export CODEX_API_KEY=your_api_key

codex exec --json \
  -o codex-summary.txt \
  "审查当前仓库，并总结最主要的 3 个风险点"
```

这一段里，三个关键点分别是：

- CODEX_API_KEY：给非交互执行做认证。
- --json：让标准输出更适合被脚本读取。
- -o codex-summary.txt：把最后一条自然语言总结单独写到文件里。

26.2 一个适合 CI 的思路

你可以把 codex exec 放到下面这些位置：

- 拉取请求（PR）创建后做风险总结；
- 每日巡检时总结仓库变化；
- 发布前做一轮只读式代码审查；
- 自动生成本次改动摘要。

26.3 GitHub Actions 风格示例

```
name: codex-check

on:
  pull_request:

jobs:
  codex-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Codex CLI
        run: npm i -g @openai/codex

      - name: Run Codex Exec
        env:
          CODEX_API_KEY: ${{ secrets.CODEX_API_KEY }}
        run: |
          codex exec --json -o codex-summary.txt \
            "审查当前改动相对基线分支的差异，并总结主要风险"
```

26.4 CI 里最容易踩的坑

- 直接照搬交互式习惯，没有切到 `codex exec`。
- 忘了提供 API 密钥。
- 没有考虑当前目录是不是 Git 仓库。
- 只看 stdout，没有把最终摘要单独落文件。

注意

自动化场景更适合做“总结、分类、只读式检查”这类任务。真正会改很多文件的流程，先在本地跑稳，再考虑自动化。

小结

CI 集成的关键不是把一切自动化，而是先找到最适合自动化的那一层：总结、审查、提示、归档。

27

附录：真实排错案例

这一节学完你能做什么

学完这一节，你会看到一条更接近真实开发现场的排错路径，不只是“报错了怎么办”，而是“遇到一个具体报错时，怎样让 Codex 帮你缩小范围”。

这一页故意写成案例，而不是抽象原则。因为新手最容易卡住的点，不是“不知道要排错”，而是不知道**第一句该怎么问**、**第二步该看什么**。

27.1 案例背景

假设你在一个前端项目里运行开发命令时，终端出现类似错误：

```
TypeError: Cannot read properties of undefined (reading 'map')
```

你目前只知道三件事：

- 页面白屏了；
- 报错和某个列表渲染有关；
- 你还没定位到具体文件。

27.2 第一轮，不要急着让它直接修

先这样问：

```
我刚遇到一个运行时报错：
TypeError: Cannot read properties of undefined (reading 'map')

请先不要直接修改代码。
先帮我做三件事：
1. 解释这个错误最常见的原因
2. 告诉我应该先检查哪些类型的文件
3. 给我一个从快到慢的排查顺序
```

这一轮的目标不是立刻修复，而是让 Codex 先帮你把问题分类。通常它会给出这种方向：

- 某个数组值还没准备好，就提前调用了 `map`；
- 接口返回为空，或者初始状态不是数组；
- 某个组件 props 没传到位。

27.3 第二轮，把范围压到文件级别

等第一轮拿到方向以后，再追问：

```
请根据当前项目代码，帮我定位最可能出现这个问题的文件。
优先找：
1. 渲染列表的组件
2. 初始化 state 的地方
3. 拉取接口数据并赋值的地方
```

这一步很关键。因为“抽象原因”到这里才真正开始落地到代码。

27.4 第三轮，让它给最小修复方案

等你已经知道大概是哪几个文件以后，再说：

```
现在请给我一个最小修复方案。
要求：
1. 先说明为什么会报错
2. 只改最必要的地方
3. 不要顺手重构
4. 修改后告诉我怎么验证
```

27.5 这类案例里，Codex 最值得帮你的地方

不是“替你秒修”，而是帮你更快完成下面这些动作：

- 把报错归类；
- 找最可能的文件；
- 给出从小到大的排查顺序；
- 限制改动范围；
- 补验证建议。

27.6 如果它一上来就想改很多文件怎么办

立刻把范围收回来：

```
请先不要做大范围修改。只给我最小修复方案，并说明如果只改一个文件，最可能改哪一个。
```

27.7 这页案例真正想教你的东西

不是这个错误本身，而是一种顺序：

1. 先解释报错；
2. 再定位文件；
3. 再最小修复；
4. 最后验证。

提示

当你不知道该不该马上让 Codex 动代码时，默认先让它解释和定位。先把问题缩小，再让它改，几乎总是更稳。

小结

真实排错案例的核心，不是记住某一种报错，而是记住排错顺序：先分类，后定位，再最小修复，最后验证。

28

附录：练习题页

这一节学完你能做什么

学完这一节，你可以把前面学到的用法真正练一遍，而不是只停留在“我看懂了”。

这一页给你几道适合新手的练习题。最好找一个公开小仓库，或者你自己的练手仓库，按顺序做。
如果你还没看上一页“练习题怎么看算合格”，建议先扫一眼；做题时你会更清楚什么叫真的落地。

28.1 练习一：找到阅读入口

目标：让 Codex 帮你给出项目阅读顺序。
要求：

- 问出入口文件；
- 问出核心目录；
- 问出推荐阅读顺序；
- 输出里必须包含具体文件路径。

自检点：

- 结果有没有落到具体路径；
- 结果是不是按新手视角组织的；
- 你能不能根据它给的顺序自己开始读代码。

28.2 练习二：定位一个功能

目标：找出某个功能的入口和调用链。
要求：

- 自己先选一个功能，比如登录、上传、搜索、列表渲染；

- 让 Codex 告诉你涉及哪些文件；
- 让它解释调用链，不要只列文件名。

自检点：

- 它有没有解释“文件之间怎么串起来”；
- 你能不能据此自己继续往下追。

28.3 练习三：做一个低风险小改动

目标：让 Codex 改一个你敢接受的小地方。
要求：

- 只改文案、说明或一处很小的逻辑；
- 明确要求“不要改业务逻辑”或“不要顺手重构”；
- 修改后让它列出改动文件和验证方法。

自检点：

- 改动范围有没有被控制住；
- 它有没有解释为什么这么改；
- 你有没有自己看 `/diff`。

28.4 练习四：做一次代码审查

目标：把代码审查流程走一遍。
建议动作：

```
/review
/diff
```

自检点：

- 它有没有优先指出真正的问题，而不是只给表扬；
- 你能不能分清“高风险问题”和“可选优化”；
- 你有没有把它的审查结果和实际 diff 对照起来看。

28.5 练习五：写一条更好的提示词

目标：练习把模糊需求改成清晰需求。

做法：

- 先写一句模糊请求，比如“帮我优化一下这里”；
- 再自己把它改写成包含目标、范围、限制、输出格式的版本；
- 然后分别交给 Codex，看结果差异。

自检点：

- 改写后的结果是不是明显更落地；
- 你是不是更容易继续追问；
- 结果里是不是少了空话，多了具体文件和动作。

提示

练习题不要一次做太多。对新手来说，最有效的顺序通常是：练习一、练习二、练习三。先把“理解、定位、小改动”这三件事练熟。

小结

练习题页的重点不是难度，而是帮助你把教程里的方法变成自己的操作习惯。

29

附录：AGENTS.md 示例

这一节学完你能做什么

学完这一节，你会看到一份可以直接改成自己仓库版本的 AGENTS.md 样板，也会知道为什么这个文件对长期使用 Codex 很重要。

前面已经提到，AGENTS.md 的作用是把重复规则沉淀下来。你可以把它理解成“这个仓库专属的长期系统提示词”。这样一来，Codex 每次进入仓库时都更容易知道：

- 项目怎么跑；
- 测试怎么跑；
- 哪些目录能改，哪些别碰；
- 交付结果时你最看重什么。

29.1 什么样的内容最值得写进去

最值得沉淀的，通常是下面这些信息：

- 仓库结构和重点目录说明；
- 安装、启动、测试、静态检查的标准命令；
- 改动边界，比如哪些文件是生成的、哪些目录不要动；
- 代码风格、提交前检查、输出格式要求；
- 完成任务后应该如何总结和汇报。

29.2 一份适合中小型项目起步的样板

```
# AGENTS.md

## 项目概览
```

```
- 这个仓库包含前端、后端和共享工具模块。
- 改动的首要目标是：范围小、容易评审、方便验证。
- 在做较大改动之前，优先先阅读现有代码和测试。

## 仓库结构

- `frontend/`：界面和页面代码
- `backend/`：接口处理和业务逻辑
- `shared/`：前后端共用代码
- `scripts/`：本地维护脚本
- `tests/`：自动化测试
- `docs/`：项目文档

## 工作规则

- 修改前先确认当前目录和相关文件。
- 优先做小而明确的改动，不要轻易做大范围重构。
- 除非任务明确要求，否则不要修改生成文件。
- 除非确有必要，否则不要重命名文件或移动模块。
- 如果需要高风险或破坏性改动，先说明原因再执行。

## 常用命令

- 安装依赖：`npm install`
- 启动开发环境：`npm run dev`
- 运行单元测试：`npm test`
- 运行静态检查：`npm run lint`
- 构建项目：`npm run build`

## 改动原则

- 保持现有代码风格和项目约定。
- 不要顺手修改无关文件。
- 只有在代码本身不够直观时才添加注释。
- 处理问题时优先选择最小且正确的改动。

## 验证要求

- 改完代码后，先执行最小但相关的验证步骤。
- 如果任务影响了应用行为，尽量运行相关测试。
- 如果有必须做但无法运行的检查，请在最终总结里明确说明。
```

```
## 输出要求

- 用通俗的话总结这次改了什么。
- 如果任务不算特别简单，请列出改动过的文件。
- 说明已经做了哪些验证。
- 如果还有剩余风险或后续检查项，也请写出来。

## 默认策略

- 遇到模糊需求时，先澄清问题或先给一个简短计划。
- 遇到较大任务时，先拆成更小的步骤。
- 如果用户还在熟悉代码库，优先先解释，再修改。
```

29.3 如果你的仓库更复杂，可以再加什么

如果是团队项目，还可以继续往里补：

- 分支策略，比如不要直接动主分支；
- 数据库迁移规范；
- 前后端联调顺序；
- 哪些命令耗时长，不要默认跑；
- 哪些目录包含敏感信息或外部生成产物。

29.4 写 AGENTS.md 的一个实用原则

不要把它写成“项目说明书大全”。最实用的 AGENTS.md，通常只保留 **Codex 真的会反复用到的规则**。如果一条说明不会影响它怎么读代码、怎么改代码、怎么验证结果，那就不一定要放进来。

提示

一个好用的判断标准是：如果你在三次任务里都重复说过同一句话，那它大概率就该进 AGENTS.md 了。

小结

AGENTS.md 的核心作用，不是展示项目背景，而是把“怎么做事”固定下来。写得越贴近真实工作流，Codex 后面越省心。

30

附录：版本变更提示框

这一节学完你能做什么

学完这一节，你会知道当教程、截图和你本地命令不一致时，应该优先检查哪些地方。

版本提示

- 安装方式可能变化：例如 npm 包名、推荐安装命令、首次登录入口。
- 登录方式可能变化：浏览器登录、设备码登录、API 密钥登录的推荐路径可能调整。
- 斜杠命令可能变化：命令名、输出内容和支持范围都有可能随版本演进。
- `exec` 和 `review` 的参数可能增加或调整。
- 沙箱和审批策略的默认行为可能变化，尤其是新版本加入新策略时。
- 最可靠的检查方式，始终是你本地的 `codex --help` 和对应子命令帮助。

30.1 最推荐的检查顺序

1. 先看 `codex --help`；
2. 再看 `codex <subcommand> --help`；
3. 再回到官方文档页面查对应章节；
4. 最后再参考博客、截图、旧教程。

30.2 哪些地方最容易发生版本差异

- 首次登录流程；
- 斜杠命令列表；
- `codex exec` 的自动化参数；
- `review` 的输出样式；

- 审批与沙箱策略的默认组合。

小结

当版本变化发生时，不要先怀疑自己记错了，也不要先怀疑教程错了。先看本地 help，这是最快的判断方法。

31

FAQ

这一节学完你能做什么

学完这一节，你会对几个最常见的新手问题有直接答案，尤其是“上下文窗口超了怎么办”这种很容易卡住的问题。

31.1 Q1：如果上下文窗口超了，应该怎么办？

最实用的做法不是“硬塞更多内容”，而是**主动压缩上下文**。可以按这个顺序处理：

1. 先缩小任务范围，只围绕一个功能、一个报错或一组文件继续问。
2. 用 `/compact` 压缩长会话，把已经确认的背景收成短摘要。
3. 用 `/mention` 或直接点名文件路径，只把真正关键的文件拉进当前上下文。
4. 如果当前话题已经太散，直接开一轮新会话，并把上一轮结论用 3 到 5 句话重新总结后贴进去。

提示

判断标准很简单：如果你已经开始反复贴长日志、长代码、长历史对话，就说明该收缩上下文了。先总结，再继续，通常比硬接着聊更稳。

31.2 Q2：它回答得很空泛，是模型不行吗？

大多数时候不是。更常见的原因是任务太宽、目标不够具体，或者没有要求它按文件、按步骤、按输出格式回答。先把问题缩到“哪个功能、哪几个文件、最后要什么格式”，效果通常会马上变好。

31.3 Q3：什么时候该开新会话，而不是一直接着问？

当你发现当前会话已经同时混进了多个目标，比如“看项目结构、处理报错、改文案、做 CI”，这时就该拆。一个会话尽量只服务一个主目标，后续会更稳，也更容易复盘。

31.4 Q4：会话到底怎么切？能像别的工具那样在多个会话间切来切去吗？

可以，但要分清楚是哪一种“切”。

- 想在当前终端里立刻开一轮新的：用 `/new`
- 想清屏后再开新的：用 `/clear`
- 想切回某个旧会话：用 `/resume`
- 想保留旧会话内容，但分一条新线：用 `/fork`

如果你期待的是像某些工具那样，在同一界面里长期挂着多个会话标签页来回切换，那 Codex CLI 的主思路不是这个。它更像是：**把之前的会话记录起来，需要时再恢复、分叉或新开**。

31.5 Q5：文档里是不是还有命令没讲？

现在这份长版里已经补上了**完整命令参考**，包括顶层命令、常见子命令，以及高频全局参数怎么读。
如果你只是第一次上手，还是建议先走主线章节；如果你准备查命令全景、补边角知识、对照本地版本，直接去后面的“附录：完整命令参考”会更高效。

31.6 Q6：它改完代码后，我最少要做什么检查？

如果你只做最低限度检查，最少做三件事：

1. 看 `/diff`，确认改动范围对不对；
2. 跑最相关的测试、构建或静态检查；
3. 自己走一遍最关键的用户路径。

如果时间允许，再补一次 `/review` 会更稳，因为它更容易提前指出行为风险和漏掉的测试。

小结

FAQ 里最值得先记的一条就是：上下文一旦开始发散，就收缩任务、压缩会话、必要时新开一轮。不要指望靠堆更多上下文来解决所有问题。

32

附录：完整命令参考

这一节学完你能做什么

学完这一节，你会对当前版本 Codex CLI 的顶层命令有完整地图，也知道每个命令大概该在什么场景下用。你不用把它全背下来，但至少查的时候不会迷路。

这一节根据你本机 `codex --help` 以及对应子命令帮助整理。它的目标不是把帮助输出原样抄一遍，而是把“**命令是干什么的、什么时候值得用、哪些参数最关键**”讲清楚。另一个容易混淆的点也先说清楚：**这一节讲的是 CLI 顶层命令和子命令**。会话里的斜杠命令，比如 `/status`、`/review`、`/compact`，前面已经单独讲过了。

32.1 先看顶层命令总览

命令	它是干什么的	你大概什么时候会用到
`codex [PROMPT]`	启动交互式会话	日常本地开发、连续追问、边看边改时最常用
`codex exec`	非交互执行代理任务	脚本、CI、一次性任务、结构化输出
`codex review`	非交互代码审查	想直接对未提交改动、分支差异或某个 commit 做代码审查
`codex login`	管理登录	首次登录、切换认证方式、检查登录状态
`codex logout`	清掉本地认证	需要登出、换账号、清理本机凭据时
`codex mcp`	管理外部 MCP 服务	给 Codex 接外部工具、外部数据源或第三方服务时
`codex mcp-server`	把 Codex 自己作为 MCP 服务暴露出去	做更底层的集成、桥接或工具链对接时
`codex app-server`	启动或辅助应用服务端相关工具	更偏集成、协议生成、IDE 或远端应用服务端场景
`codex completion`	生成命令行补全脚本	想让 Bash / Zsh / Fish 里补全更顺手时
`codex sandbox`	手动在 Codex 提供的沙箱里跑命令	想测试沙箱行为、单独验证受限执行环境时

命令	它是干什么的	你大概什么时候会用到
`codex debug`	调试工具集合	排查应用服务端或更底层行为时
`codex apply`	把任务产出的 diff 应用到本地工作树	你已经有任务结果，只想把补丁应用到仓库里时
`codex resume`	恢复旧交互会话	不想重新讲背景，想接着上次继续干
`codex fork`	从旧会话分叉出一条新线	想保留原结论，再试另一种思路时
`codex cloud`	浏览或提交 Codex Cloud 任务	用云端任务池、云端执行、再把结果落回本地时
`codex features`	查看或切换功能开关	想检查某个功能开关是否启用，或手动开关实验特性
`codex help`	查看帮助	忘了命令、参数、子命令时最快的入口

提示

如果你现在只想把主线抓住，先盯住 8 个：`codex`、`exec`、`review`、`login`、`logout`、`apply`、`resume`、`fork`。剩下那些命令，更多是扩展、集成或调试层面的能力。

32.2 第一组：主线工作流命令

`codex [PROMPT]`：这是默认入口。你不写子命令时，Codex 会直接进入交互式会话。它最适合“边问边追、边看边改”的工作方式。你甚至可以把第一句需求直接作为参数带进去：

```
codex
codex "先帮我梳理当前仓库入口、核心目录和推荐阅读顺序"
```

这条命令支持最多的“会话级参数”，比如选模型、选沙箱、选审批策略、切配置档、加搜索、加额外可写目录等。如果帮助里说这些选项会转发给交互式 CLI，意思就是这些参数大多会作用在这一轮交互式 CLI 里。

`codex exec [PROMPT] [COMMAND]`：这是非交互入口。它适合脚本、CI、批量总结、结构化输出，不适合那种需要你来回追问很多轮的任务。
它当前还有两个子命令：

- `codex exec resume`：在非交互模式下恢复旧会话，并追加一段提示词。
- `codex exec review`：在非交互模式下做代码审查，适合脚本化调用。

最常见的几类参数是：

- `--json`：把事件流输出成 JSONL，方便程序读取；
- `-o` / `--output-last-message`：把最后一条消息单独写到文件；
- `--output-schema`：要求最终输出符合某个 JSON Schema；
- `--skip-git-repo-check`：允许不在 Git 仓库里运行；
- `--ephemeral`：本轮不把会话文件持久化到磁盘。

你可以这样用：

```
codex exec --json -o summary.txt "总结当前改动最主要的三个风险点"
```

`codex review [PROMPT]`：这是非交互代码审查入口。你可以把它理解成“直接跑一轮审查模式”，不用先进入交互界面。
最关键的三个定位参数是：

- `--uncommitted`：看当前已暂存、未暂存和未跟踪的改动；
- `--base <BRANCH>`：拿当前改动和某个基线分支对比；
- `--commit <SHA>`：只审查某个 commit 引入的改动。

```
codex review --uncommitted
codex review --base main
```

如果你要在脚本里读取结果，又想保留代码审查这层语义，也可以考虑 `codex exec review`，因为它额外支持 `--json`、`-o`、`--ephemeral` 这类非交互参数。

`codex login`：管理登录。它本身最重要的两个场景，一个是第一次登录，一个是切换认证方式。
当前最值得知道的入口有三个：

- `codex login`：走默认登录流程；
- `codex login status`：检查当前登录状态；
- `codex login --with-api-key`：从标准输入读 API 密钥；
- `codex login --device-auth`：适合远端 Linux、SSH 或无浏览器环境。

```
codex login
codex login status
printenv OPENAI_API_KEY | codex login --with-api-key
```

codex logout：清掉当前本地认证信息。它的用途很直白，就是登出。
最常见的使用场景是：

- 你准备换账号；
- 你想清理本机认证缓存；
- 你怀疑本地认证状态出了问题，想重新登录。

codex apply <TASK_ID>：把某个任务产出的 diff 应用到你本地工作树。帮助里说得很直接，它底层会把结果作为 `git apply` 打到工作树里。
这条命令特别适合“结果已经有了，但我现在只想把补丁应用到本地”这种场景。你需要给它一个 `TASK_ID`，所以它不是日常第一次上手就会高频用到的入口。

codex resume [SESSION_ID] [PROMPT]：恢复旧交互会话。默认会打开一个选择列表；如果你只想接着最近一次，直接加 `--last`。
它还支持：

- `--all`：显示全部会话，而不是只看当前工作目录附近的；
- `--include-non-interactive`：把非交互会话也纳入候选；
- 交互会话那批常见参数，比如 `-m`、`-s`、`-a`、`--search`、`--add-dir`。

```
codex resume --last
codex resume 123e4567-e89b-12d3-a456-426614174000 "继续上次的排查，重点看缓存层"
```

codex fork [SESSION_ID] [PROMPT]：从旧会话分叉。它和 `resume` 的差别不是“能不能回到旧上下文”，而是“保不保留原线继续存在”。
当你想把旧结论留着，同时试另一个假设、另一个方案、另一个排障方向时，用它最顺。

```
codex fork --last
codex fork 123e4567-e89b-12d3-a456-426614174000 "这次改从状态同步问题的角度再看一遍
"
```

codex help [COMMAND]：最朴素，但也最可靠。你忘了什么，就查什么。
如果你只记一条“不会过时”的建议，那就是：

```
codex help
codex help exec
codex exec --help
```

32.3 第二组：扩展、集成和云端命令

codex mcp：管理外部 MCP（Model Context Protocol）服务。你可以把它理解成“给 Codex 接外部能力”的入口。
当前子命令有 6 个，分工很清楚：

子命令	用途
mcp list	查看当前配置过的 MCP 服务；加 --json 可以输出方便程序读取的结果
mcp get <NAME>	看某个 MCP 服务的具体配置；也支持 --json
mcp add <NAME> --url <URL>	添加一个基于 HTTP 的 MCP 服务
mcp add <NAME> -- <COMMAND>...	添加一个通过 stdio 启动的 MCP 服务，可配 --env KEY=VALUE
mcp remove <NAME>	删除一个 MCP 服务配置
mcp login <NAME>	对某个 MCP 服务做 OAuth 登录；可加 --scopes
mcp logout <NAME>	清掉某个 MCP 服务的认证状态

```
codex mcp list
codex mcp add my-server --url https://example.com/mcp
codex mcp add local-tools -- /path/to/mcp-server --flag
```

codex mcp-server：把 Codex 自己作为 MCP 服务启动起来，当前帮助里写的是 stdio 模式。
这条命令对普通日常开发者不算高频，更适合做工具链集成、桥接、IDE 插件、或让别的客户端把 Codex 当服务来调。

codex app-server：实验性命令。它既能启动应用服务端，也能做协议相关生成。
顶层最关键的是：

- --listen：选择监听端点，默认是 stdio://，也可以走 websocket；
- 一组 --ws-* 参数：控制 websocket 鉴权方式、token 文件、签名配置等；
- generate-ts：生成 TypeScript 绑定；
- generate-json-schema：生成 JSON Schema。

```
codex app-server --listen stdio://
codex app-server generate-ts --out ./generated
codex app-server generate-json-schema --out ./schema
```

codex cloud：实验性云端任务入口。它的定位不是本地交互，而是“看云端任务、提云端任务、把结果拉回来”。
当前子命令有 5 个：

子命令	用途
cloud exec --env <ENV_ID> [QUERY]	提交一个新的 Cloud 任务；可加 --attempts 和 --branch
cloud status <TASK_ID>	看某个 Cloud 任务的状态
cloud list	列任务；支持 --env、--limit、--cursor、--json
cloud diff <TASK_ID>	看某个任务的 unified diff；可指定 --attempt
cloud apply <TASK_ID>	把某个任务的 diff 应用到本地；也可指定 --attempt

这条命令最像“云端版的任务队列”。如果你当前主要是在本地终端里工作，可以先知道有它，不一定马上会高频使用。

codex features：查看或切换功能开关。它更像一个“功能开关面板”，不是日常主线命令。
当前子命令是：

- features list：看已知功能开关以及当前启用状态；
- features enable <FEATURE>：在 config.toml 里开启一个功能；
- features disable <FEATURE>：在 config.toml 里关闭一个功能。

```
codex features list
codex features enable unified_exec
codex features disable unified_exec
```

32.4 第三组：环境、沙箱和调试命令

codex completion [SHELL]：生成命令行补全脚本。帮助里支持的命令行环境包括 bash、zsh、fish、powershell、elvish。
这条命令本身不直接改你的命令行配置，它只是把补全脚本打印出来或生成出来，后续通常要你自己接到对应配置里。

```
codex completion bash
codex completion zsh
```

codex sandbox：手动在 Codex 提供的沙箱里跑命令。这个命令不是“让代理在沙箱里工作”的入口，而是你自己显式去调用沙箱能力。
当前有 3 个平台子命令：

子命令	用途
sandbox linux [COMMAND]...	在 Linux 沙箱里执行命令；支持 --full-auto
sandbox macos [COMMAND]...	在 macOS Seatbelt 下执行；额外支持 --log-denials
sandbox windows [COMMAND]...	在 Windows restricted token 沙箱下执行

因为你现在主要写 Linux 教程，所以最值得记的是：

```
codex sandbox linux --full-auto bash -lc 'pwd && ls'
```

它更适合“验证沙箱行为本身”，而不是替代你平时直接运行 codex。

codex debug：调试工具集合。它当前主要围绕应用服务端调试。
当前层级是：

- debug app-server：应用服务端调试工具入口；
- debug app-server send-message-v2 <USER_MESSAGE>：向调试目标发一条消息。

这类命令更偏内部调试、协议调试、集成排障。对绝大多数“我只是想写代码、改文档、做代码审查”的使用者来说，不属于第一层必学内容。

32.5 高频全局参数怎么读

很多帮助里你会反复看到同一批参数。它们不是“每个命令都支持”，但确实是一批**高频复用参数**。先认熟这些，帮助阅读速度会快很多。

会话类命令常见参数

参数	作用
-c, --config <key=value>	临时覆盖 ~/.codex/config.toml 的某个配置项
--enable / --disable <FEATURE>	临时打开或关闭某个功能开关
-m, --model <MODEL>	指定本轮会话要用的模型
--oss	改走本地开源模型提供方，而不是默认的云端提供方
--local-provider <OSS_PROVIDER>	细分本地模型提供方，比如 lmstudio 或 ollama
-p, --profile <CONFIG_PROFILE>	从配置文件里选一个配置档
-s, --sandbox <SANDBOX_MODE>	选沙箱级别：read-only、workspace-write、danger-full-access
-a, --ask-for-approval <APPROVAL_POLICY>	选审批策略，比如 untrusted、on-request、never
--full-auto	一个比较省事的默认组合，通常等于更自动的审批 + workspace-write
--dangerously-bypass-approvals-and-sandbox	完全跳过审批和沙箱，风险极高，只适合你自己非常清楚边界时
-C, --cd <DIR>	指定工作根目录
-i, --image <FILE>	把图片作为初始输入附上
--search	给模型打开联网搜索能力
--add-dir <DIR>	在主工作目录之外，再额外加一个可写目录
--no-alt-screen	关闭备用屏模式，保留终端滚动历史

非交互和自动化高频参数

参数	作用
--skip-git-repo-check	允许不在 Git 仓库里运行
--ephemeral	不把会话文件持久化到磁盘

参数	作用
`--json`	把事件或结果输出成 JSONL，方便脚本处理
`-o, --output-last-message <FILE>`	把最后一条消息单独写到文件
`--output-schema <FILE>`	要求最终输出符合某个 JSON Schema
`--uncommitted`	审查当前未提交改动
`--base <BRANCH>`	审查当前改动相对某个基线分支的变化
`--commit <SHA>`	审查某个 commit 引入的变化
`--title <TITLE>`	给审查摘要一个标题

注意

帮助里经常会重复出现 `-c`、`--enable`、`--disable`。它们不是“这条命令有什么特别功能”，而是整套 CLI 共用的一种配置入口。真正决定场景差异的，通常还是命令本身和那条命令独有的参数。

32.6 最后给一个查命令的顺序

当你忘了某个命令具体怎么用，最稳的顺序通常是：

1. 先看这节，判断它属于哪一类命令；
2. 再跑 `codex help <command>` 或 `codex <command> --help`；
3. 如果它还有子命令，再继续看 `codex <command> <subcommand> --help`；
4. 最后再结合你当前项目场景决定要不要真的执行。

小结

真正的“完整命令参考”不是把帮助背下来，而是知道三件事：这条命令在哪一层、适合什么场景、我下一步该查哪一级 help。只要这三件事清楚了，你基本就不会在命令树里迷路。

33

官方资源与版本说明

这一节学完你能做什么

学完这一节，你会知道这份教程背后的主要参考来源，也知道当本地行为和教程不一致时应该信谁。

这份教程的关键事实主要来自两类来源：

- 本地 CLI 帮助输出：`codex --help`、`codex exec --help`、`codex review --help`、`codex login --help` 等。
- OpenAI 官方 Codex 文档。

建议优先查看这些官方页面：

- CLI 总览：https://developers.openai.com/codex/cli
- 认证：https://developers.openai.com/codex/auth
- 审批与安全：https://developers.openai.com/codex/agent-approvals-security
- 命令行选项：https://developers.openai.com/codex/cli/reference
- 斜杠命令：https://developers.openai.com/codex/cli/slash-commands
- 非交互模式：https://developers.openai.com/codex/noninteractive
- 最佳实践：https://developers.openai.com/codex/learn/best-practices

33.1 当教程和你本地行为不一致时怎么办

优先级建议这样排：

1. 先看你本地的 `codex --help`；
2. 再看对应的官方文档页面；
3. 最后再看第三方博客、帖子或旧截图。

进阶阅读

原因很简单：Codex CLI 更新比较快，而第三方教程经常会落后于当前版本。

小结

真正可依赖的来源，始终是“你机器上的帮助信息 + OpenAI 官方 Codex 文档”。这两者要优先于任何二手资料。

34

一页式总结页

1. 先做什么
进入项目目录，确认位置没错。

2. 先跑什么
先看 `codex --help`，再登录，再做最小验证。

3. 日常怎么用
交互式用 `codex`，边问边收紧范围。

4. 改完怎么查
先 `/review`，再 `/diff`，再自己验证。

5. 权限怎么选
本地优先 `workspace-write`，别急着全放开。

6. 一句话记忆
说清楚任务，小步推进，看状态，审改动，做验证。